

FACULTY OF ENGINEERING
DEGREE ENGINEERING
DEPARTMENT OF COMPUTER & INFORMATION TECHNOLOGY
Semester – 5th



Computer Networks (BE103404)

Faculty Name – Radhika Gokani

Introduction

❖ OSI MODEL

The OSI (Open Systems Interconnection) Model describes how networked systems communicate. It is a theoretical framework for mapping how applications and devices link and send data over a web of connections.

The OSI Model is called a “reference model” because it is rarely used in practical applications. Instead, it is most helpful in describing network protocols and services. The OSI Model is most often used to understand the theories behind networked communication concepts and elements. Some use it to troubleshoot problems, narrowing a potential issue to a specific layer of the model.

OSI layers are, indeed, fundamental to the OSI Model. The framework is a stack of seven layers, each performing a specific function, yet working together to transmit data from one networked device to another. The 7 layers of OSI model ensure the interoperability of different devices and technologies from various vendors.

- [The Physical Layer](#)
- [The Data Link Layer](#)
- [The Network Layer](#)
- [The Transport Layer](#)
- [The Session Layer](#)
- [The Presentation Layer](#)
- [The Application Layer](#)

OSI Model		
DATA	LAYER	FUNCTIONS
DATA	APPLICATION	Network process to application
DATA	PRESENTATION	Data representation and encryption
DATA	SESSION	Interhost communication
SEGMENTS	TRANSPORT	End -to-end connection and reliability
PACKETS	NETWORK	Path Determination and IP (local addressing)
FRAMES	DATA	MAC and LLC (physical addressing)
BITS	PHYSICAL	Media signal and transmission



7 Layers of the OSI Model



7. Application

- End User layer
- HTTP, FTP, IRC, SSH, DNS



6. Presentation

- Syntax layer
- SSL, SSH, IMAP, FTP, MPEG, JPEG



5. Session

- Synch & send to port
- API's, Sockets, WinSock



4. Transport

- End-to-end connections
- TCP, UDP



3. Network

- Packets
- IP, ICMP, IPSec, IGMP



2. Data Link

- Frames
- Ethernet, PPP, Switch, Bridge



1. Physical

- Physical structure
- Coax, Fiber, Wireless, Hubs, Repeaters

1. Physical Layer

The first OSI model layer describes the physical connections between devices in a network. Electrical, optic, or electromagnetic signal data moves from device to device through the physical infrastructure defined by this layer.

Elements of the OSI layer model's physical layer include:

- Electrical, mechanical, and physical systems, parts, and devices
 - Specifications such as cable size, signal frequency, voltages, etc.
 - How a network is configured, such as a bus, with devices arranged in a line with many stops, a star, with a central device surrounded by an array of devices, a ring with devices in a connected circle, a mesh of woven interconnections, and more
 - Communication modes such as one-way simplex, two-way full duplex, or half duplex, with messages going back and forth, but only one direction at a time
 - Data transmission performance, such as bit rate, referring to how much data is moving through a connection at a given time, and bit synchronization to align the clocks of each device for accurate data transmission.
-
- Modulation, switching, and interfacing with the physical transmission medium
Common protocols for connections, including Wi-Fi, Ethernet, and others
Hardware, including networking devices, antennas, cables, modems, and intermediate devices such as repeaters and hubs

2. The Data Link Layer

The second OSI model layer describes data transmission between network devices. Data is sent in packets within a frame of special bit patterns or codes that indicate the beginning and end of each packet. These packets move through switches that route information through the physical layer from one physical location to another.

The OSI data link layer has two sublayers:

- The Logical Link Control (LLC) sublayer manages flow and error controls to ensure accurate data transmission between the network devices.
- The Media Access Control (MAC) sublayer manages access and permissions for transmitting data between devices. The function of this sublayer is to manage which device controls a channel, moment to moment.

3. The Network Layer

The third of the OSI layers organizes and transmits data between multiple networks. Network layer hardware includes routes, bridge routers, 3-layer switches, and protocols such as Internet (IPv4) Protocol version 4 and Internet Protocol version 6 (IPv6).

This layer routes data via the shortest or fastest physical path, working around traffic controls, congestion, broken links, service priority, and more. It handles addressing logic to distinguish between the source and destination networks. It divides data into packets to send and then reassembles them at their destination.

4. The Transport Layer

This layer deals with sending and delivering of complete and reliable data from one device to another through a network or between networks. Common transport layer protocols include the Transmission Control Protocol (TCP) for connection-oriented data transmission and the User Datagram Protocol (UDP) for connectionless data transmission.

Some of the essential functions in this layer include:

- Error control, flow control, and congestion control are ways to keep track of data packets, check for errors and duplication, and then to resend if there is an error or failure.
- Service-point addressing ensures that data is delivered to the correct protocol, identified by a port number.
- Packet segmentation and reassembly are processes for dividing data and sending it sequentially, then rechecking it at its destination for integrity and accuracy.

5. The Session Layer

This OSI Model layer concerns itself with managing session links between network devices with a specific beginning and end. The essential functions include establishing a link to start the session, authenticating senders and receivers, authorizing communications between devices and apps, maintaining the session, and terminating the connection, which are the key functions.

A common session type is when an internet user visits and browses a website for a specific period.

An essential concept in this layer is synchronization, in which checkpoints during the session ensure a coordinated data flow that is free of unplanned breaks or data loss.

Common OSI session layer protocols include:

- Remote procedure call protocol (RPC)
- Point-to-Point Tunneling Protocol (PPTP)
- Session Control Protocol (SCP)
- Session Description Protocol (SDP)

6. The Presentation Layer

Data flows as packets of code, often encrypted, through networks. Presenting it in a useful format is the focus of the sixth OSI model layer. The presentation layer deals with:

- Data conversion
- Character code translation
- Data compression
- Encryption and decryption

To send text across a network, the characters of the alphabet convert via a character coding system, such as the American Standard Code for Information Interchange (ASCII) or Extended Binary Coded Decimal Interchange Code (EBCDIC) that is then encrypted and compressed and sent over the network. On the receiving end, the process reverses. Different kinds of data get translated into different format codes.

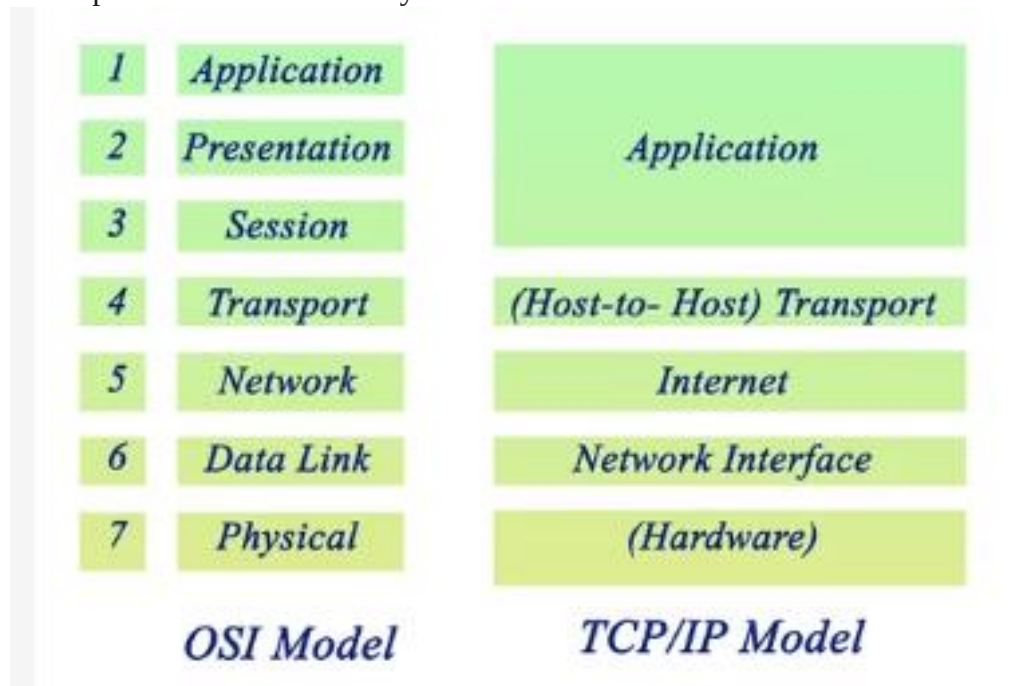
7. The Application Layer

The top layer of the OSI 7 layer model is the application layer. It is how a user application, such as a website, browser, email, instant messaging, file transfers, or voice-over IP, interfaces with the network. Think of it as the window for accessing the network to send or display data, which can be anything from a picture of your pet cat to a database of statistics to a voice message.

It facilitates networking requests, determines resource availability, synchronizes communication, and manages application-specific networking requirements. The application layer also identifies constraints at the application level, such as those associated with authentication, privacy, quality of service, networking devices, and data syntax.

OSI vs. TCP/IP

The OSI vs. TCP/IP debate never really gained traction in the industry. Vendors had already invested significant resources in TCP/IP products in the ten-plus years before the launch of the OSI. They had no incentive to manage the interoperability issues with the vast choices of protocols and specifications offered by the OSI model. Additionally, the creation of the OSI Model was driven by [European political and economic interests](#) that didn't want to be dependent on technologies and equipment developed by American companies. Hence, a non-governmental organization, the International Organization for Standardization, designed the rather ponderous new model by committee.



❖ Differences between OSI and TCP/IP model

	OSI	TCP/IP
1. Layer Structure	OSI has 7 distinct layers, each with a clearly defined function.	TCP/IP has 4 layers with some combined responsibilities.
2. Protocol Dependency	OSI is protocol-independent , meaning it doesn't specify protocols — it's more a guideline.	TCP/IP is protocol-specific , with defined standards like TCP, IP, FTP, etc.
3. Modularity and Flexibility	OSI's strict modular design makes it easier to troubleshoot specific layers.	TCP/IP's loose layering allows faster and more flexible integration of protocols.
4. Real-World Adoption	OSI is mostly used for teaching and conceptual clarity.	TCP/IP is the backbone of the modern internet — globally adopted.
5. Development Sequence	OSI model was developed before the protocols .	TCP/IP protocols were developed before the model , which led to a more practical approach.
6. Reliability	OSI assigns reliability to the Transport layer only.	TCP/IP allows reliability to be handled at multiple layers , mostly via TCP in the Transport layer.

❖ TCP/IP MODEL

The **TCP/IP Model** (Transmission Control Protocol/Internet Protocol) is a *practical networking framework* developed in the 1970s by DARPA for reliable communication between computers. Unlike the OSI model, it's based directly on working protocols used in the real world. TCP/IP consists of **four layers**, each handling specific functions:

1. Application Layer

- **Purpose:** Interfaces with user applications.
- **Responsibilities:**
 - Data generation (e.g., web browsing, email, file transfer)
 - Protocols:
 - **HTTP/HTTPS** (web)
 - **FTP** (file transfer)
 - **SMTP/POP/IMAP** (email)
 - **DNS** (domain name resolution)
- **Corresponding OSI Layers:** Combines OSI's Application, Presentation, and Session layers.

2. Transport Layer

- **Purpose:** Provides reliable or unreliable data delivery.
- **Key Protocols:**
 - **TCP** (Transmission Control Protocol): Reliable, connection-oriented
 - **UDP** (User Datagram Protocol): Faster, connectionless
- **Functions:**
 - Error detection and correction
 - Flow control
 - Data segmentation and reassembly

3. Internet Layer

- **Purpose:** Handles routing and addressing.
- **Key Protocols:**
 - **IP** (Internet Protocol): Addressing and delivery of packets
 - **ICMP** (for diagnostics — ping)
 - **ARP** (Address Resolution Protocol)
- **Functions:**
 - Logical addressing
 - Routing across networks
 - Packet forwarding

4. Link Layer (or Network Access Layer)

- **Purpose:** Manages physical transmission of data.
- **Components:**
 - Device drivers
 - Network interface cards (NICs)
 - MAC addressing
- **Functions:**
 - Frame formatting
 - Error detection at hardware level
 - Access control (who can send data on the medium)



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

Faculty of Engineering (Degree)
Semester – 5th
Subject Name –Computer Networks
Subject Code - BE103404
Faculty Name – Radhika Gokani

Chapter -1
Introduction to computer networks and Internet

1. Networks Introduction to computer and Internet

Computer Networks

A computer network is a group of interconnected devices that can communicate and share resources. These devices can include computers, servers, smartphones, and other hardware. Networks can be classified based on their size and scope:

- **Local Area Network (LAN):** Covers a small geographic area, like a home, office, or building.
- **Wide Area Network (WAN):** Spans a large geographic area, often a country or continent, and connects multiple LANs.
- **Metropolitan Area Network (MAN):** Covers a city or a large campus.
- **Personal Area Network (PAN):** Connects devices within a person's workspace, typically within a range of a few meters.

Internet

The **Internet** is the largest and most well-known computer network, connecting millions of private, public, academic, business, and government networks worldwide. It uses a standard set of protocols called **TCP/IP** (Transmission Control Protocol/Internet Protocol) to ensure different networks can communicate with each other.

Key Concepts

- **Nodes and Hosts:** Devices on a network.
- **Packets:** Small units of data transmitted over a network.
- **Protocols:** Rules that govern data communication. Common protocols include HTTP (for web browsing), FTP (for file transfer), and SMTP (for email).
- **IP Address:** A unique identifier for each device on a network.
- **Routers and Switches:** Devices that direct data traffic within and between networks.

Types of Networks

- **Social Networks:** Platforms like Facebook and Twitter that connect people.
- **Mobile Networks:** Networks that provide connectivity to mobile devices.
- **Computer Networks:** Networks that connect computers and other devices for data sharing and communication.

1.1 Understanding of network and Internet

Network

A **network** is a collection of computers, servers, and other devices connected to each other to share data and resources. Networks can vary in size and scope, and they are typically categorized into different types:

- **Local Area Network (LAN):** Covers a small area like a home, office, or building. Devices in a LAN are usually connected via Ethernet cables or Wi-Fi.
- **Wide Area Network (WAN):** Spans a large geographical area, such as a city, country, or even globally. The Internet itself is a type of WAN.
- **Metropolitan Area Network (MAN):** Covers a larger area than a LAN but smaller than a WAN, such as a city or a large campus.

Internet

The **Internet** is a vast global network of networks that connects millions of devices worldwide. It is a collection of public and private networks that are interconnected using standardized communication protocols. The primary purpose of the Internet is to provide a platform for communication, collaboration, and information sharing.

Key Differences

- **Scope:** A network can be as small as two connected devices, while the Internet is a global network of interconnected networks.
- **Purpose:** Networks are designed to facilitate communication and resource sharing within a specific area, whereas the Internet connects networks globally to enable broader communication and information exchange.
- **Technology:** Networks use various technologies like Ethernet, Wi-Fi, and fiber optics, while the Internet relies on standardized protocols like TCP/IP to ensure different networks can communicate with each other.

1.2 The network edge

In computer networks, the **network edge** is the point where a device or local network connects to the internet or another external network. This is a crucial area for both performance and security. Here are some key aspects:

1. **Connection Point:** The network edge is where devices like personal computers, smartphones, and IoT devices connect to the internet¹.
2. **Security Boundary:** It acts as a security boundary, protecting the internal network from external threats.

3. **Data Processing:** Edge computing often occurs here, processing data close to its source to improve response times and reduce bandwidth usage.
4. **Components:** Common components at the network edge include routers, switches, firewalls, and modems.

1.3 The network core

The **network core**, also known as the backbone network, is the central part of a computer network that connects various subnetworks and ensures efficient data transfer between them. Here are some key aspects:

1. **Connectivity and Routing:** The core network provides connectivity and routing services between different parts of the network, ensuring data packets are efficiently forwarded from one point to another.
2. **High Speed and Reliability:** It is designed to be very fast, reliable, scalable, and secure, handling large volumes of data with minimal latency.
3. **Components:** The core network typically includes high-capacity routers and switches that manage data traffic between different network segments.
4. **Packet and Circuit Switching:** It supports both packet switching, where data is divided into packets and sent independently, and circuit switching, where a dedicated path is established for the duration of a communication session.

1.4 Understanding of Delay

In computer networks, delay refers to the time it takes for data to travel from the source to the destination. There are several types of delays that can affect network performance:

1. **Transmission Delay:** This is the time required to push all the packet's bits onto the wire. It depends on the packet's length and the transmission rate of the link. For example, if the bandwidth is 1 Mbps and the packet size is 1,000 bits, the transmission delay is 1 millisecond.
2. **Propagation Delay:** This is the time it takes for a signal to travel from the sender to the receiver. It depends on the distance between the two and the propagation speed of the medium (e.g., fiber optics, copper wire). For instance, in fiber optics, the propagation speed is approximately (2×10^8) meters per second.
3. **Queueing Delay:** This occurs when packets wait in a queue before being transmitted. It varies based on the network traffic load. High traffic can lead to longer queueing delays.
4. **Processing Delay:** This is the time taken by routers and switches to process the packet header and determine the packet's next hop. It depends on the processing power of the network devices.

1.5 Loss and Throughput in the packet

In packet-switched networks, **loss** and **throughput** are crucial metrics that impact network performance.

Packet Loss

Packet loss occurs when data packets traveling across a network fail to reach their destination. This can happen due to network congestion, faulty hardware, or other

issues. Packet loss directly reduces throughput because some data is never received and can't be counted as part of the total data transferred. Additionally, many transport layer protocols interpret packet loss as a sign of congestion and will reduce their transmission rate to avoid further congestion.

Throughput

Throughput refers to the amount of data successfully transferred from one point to another in a given time period. It is often measured in bits per second (bps). Throughput can be affected by various factors, including packet loss, network congestion, and the efficiency of the network protocols in use.

Relationship Between Loss and Throughput

The relationship between packet loss and throughput is inversely proportional. As packet loss increases, throughput generally decreases because more data needs to be retransmitted, and protocols may slow down the transmission rate to manage congestion.

1.6 switching network

In computer networks, **switching** is the process of directing data packets from one device to another within the same network or across different networks. This is done using devices called **switches**. Here are the main types of switching techniques:

1. Circuit Switching

- **Description:** Establishes a dedicated communication path between two devices for the duration of the communication session.
- **Use Case:** Traditional telephone networks.
- **Advantages:** Provides a constant data rate and predictable performance.
- **Disadvantages:** Inefficient for data transmission as the dedicated path can lead to wasted bandwidth.

2. Packet Switching

- **Description:** Data is broken into packets, each of which can take different paths to the destination. The packets are reassembled at the destination.
- **Use Case:** Most modern computer networks, including the internet.
- **Advantages:** Efficient use of network resources, supports multiple communication sessions simultaneously.
- **Disadvantages:** Potential for packet loss and delay.

3. Message Switching

- **Description:** Entire messages are sent from the source to the destination, one hop at a time. Each intermediate device stores the entire message before forwarding it.
- **Use Case:** Early data networks.
- **Advantages:** No need for a dedicated path, can handle large messages.

- **Disadvantages:** High latency due to the store-and-forward nature.

Network Switches

A **network switch** is a hardware device that connects multiple devices within a network and uses MAC addresses to forward data to the correct destination. Switches operate at the Data Link layer (Layer 2) of the OSI model

1.7 protocols layers and their service model

In computer networking, protocols are organized into layers, each with specific functions and services. The most well-known models are the **OSI (Open Systems Interconnection) model** and the **TCP/IP model**. Here's a brief overview of each layer and its service model:

OSI Model

The OSI model has seven layers, each providing specific services to the layer above it:

1. **Physical Layer**
 - **Function:** Transmits raw bit streams over a physical medium.
 - **Protocols:** Ethernet, USB.
2. **Data Link Layer**
 - **Function:** Provides node-to-node data transfer and error detection/correction.
 - **Protocols:** MAC (Media Access Control), PPP (Point-to-Point Protocol).
3. **Network Layer**
 - **Function:** Handles routing of data packets between devices across different networks.
 - **Protocols:** IP (Internet Protocol), ICMP (Internet Control Message Protocol).
4. **Transport Layer**
 - **Function:** Ensures complete data transfer with error recovery and flow control.
 - **Protocols:** TCP (Transmission Control Protocol), UDP (User Datagram Protocol).

5. **Session Layer**
 - **Function:** Manages sessions or connections between applications.
 - **Protocols:** NetBIOS, RPC (Remote Procedure Call).
6. **Presentation Layer**
 - **Function:** Translates data between the application layer and the network, including encryption and compression.
 - **Protocols:** SSL/TLS (Secure Sockets Layer/Transport Layer Security).
7. **Application Layer**
 - **Function:** Provides network services directly to end-user applications.
 - **Protocols:** HTTP (Hypertext Transfer Protocol), FTP (File Transfer Protocol), SMTP (Simple Mail Transfer Protocol).

TCP/IP Model

The TCP/IP model, which is more commonly used in practice, has four layers:

1. **Link Layer**
 - **Function:** Corresponds to the OSI's Physical and Data Link layers, handling hardware addressing and media access control.
 - **Protocols:** Ethernet, ARP (Address Resolution Protocol).
2. **Internet Layer**

- **Function:** Corresponds to the OSI's Network layer, responsible for packet forwarding and routing.
- **Protocols:** IP, ICMP.
- 3. **Transport Layer**
 - **Function:** Similar to the OSI's Transport layer, ensuring reliable data transfer.
 - **Protocols:** TCP, UDP.
- 4. **Application Layer**
 - **Function:** Combines the OSI's Session, Presentation, and Application layers, providing end-user services.
 - **Protocols:** HTTP, FTP, SMTP.

Service Models

Each layer provides services to the layer above it. For example:

- The **Network Layer** provides routing services to the **Transport Layer**.
- The **Transport Layer** provides reliable data transfer services to the **Application Layer**.

These models help standardize network communication and make it easier to develop and troubleshoot networked systems

1.8 History of the computer network

The history of computer networking is fascinating and spans several decades, marked by significant milestones and technological advancements. Here's a brief overview:

Early Developments

- **1961:** Leonard Kleinrock proposed the idea of ARPANET, one of the earliest computer networks, in his paper "Information Flow in Large Communication Nets"¹.
- **1965:** Donald Davies coined the term "packet" to describe data sent between computers over a network¹.
- **1969:** ARPANET, the first network to use packet switching, was developed. The first data transmission occurred between UCLA and the Stanford Research Institute¹.

Key Milestones

- **1973:** Robert Metcalfe developed Ethernet at Xerox PARC, which became a foundational technology for local area networks (LANs)¹.
- **1978:** Bob Kahn and Vint Cerf developed the TCP/IP protocol, which became the standard for internet communication¹.
- **1983:** ARPANET transitioned to using TCP/IP, marking the beginning of the modern internet¹.
- **1986:** The National Science Foundation Network (NSFNET) was established, providing a backbone for ARPANET and eventually replacing it.

Modern Era

- **1990s:** The World Wide Web was invented by Tim Berners-Lee, revolutionizing how information is accessed and shared².
- **2000s:** The rise of wireless networking technologies, such as Wi-Fi, further expanded the reach and convenience of computer networks



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

Faculty of Engineering (Degree)
Semester – 5th
Subject Name –Computer Networks
Subject Code - BE103404
Faculty Name – Radhika Gokani

Chapter -2
Application Layer

2. Introduction

The **Application Layer** is the topmost layer in both the OSI and TCP/IP models. It serves as the interface between the end-users and the network services, facilitating various network activities.

Functions of the Application Layer

1. **Identifying Communication Partners:** Determines the availability of communication partners for an application with data to transmit.
2. **Resource Availability:** Ensures that sufficient network resources are available for the requested communication.
3. **Synchronizing Communication:** Manages the synchronization of communication between applications.

Services Provided by the Application Layer

- **Email Services:** Facilitates email forwarding and storage.
- **File Transfer, Access, and Management (FTAM):** Allows users to access, retrieve, and manage files on remote computers.
- **Directory Services:** Provides access to global information about various objects and services.
- **Network Virtual Terminal:** Enables users to log on to a remote host by creating a software emulation of a terminal.

Protocols in the Application Layer

Some common protocols operating at this layer include:

- **HTTP/HTTPS:** Used for web browsing.
- **FTP:** Used for file transfers.
- **SMTP:** Used for sending emails.
- **DNS:** Used for resolving domain names to IP addresses.

The Application Layer is crucial as it directly interacts with the end-user and provides the necessary services to facilitate communication over the network.

2.1 Principles of computer applications

The principles of computer applications encompass several key concepts that guide the design, development, and use of applications running on computer networks:

1. Network Application Architectures

- **Client-Server Architecture:** One component acts as a client making requests, and another acts as a server providing services. Common in web applications.
- **Peer-to-Peer (P2P) Architecture:** Each component can act as both a client and a server, communicating directly with each other. Used in file-sharing applications.
- **Three-Tier Architecture:** Divides the application into client, middleware, and database components, improving performance and scalability.
- **Microservices Architecture:** Consists of small, independently deployable components (microservices) that communicate through APIs, offering flexibility and scalability¹.

2. Processes Communicating

- Communication between processes (programs or tasks) on the same or different devices is facilitated by protocols that define rules and formats for data exchange¹.

3. Application-Layer Protocols

- Define rules for data exchange between applications, including formatting, error detection, correction, and security. Examples include HTTP, HTTPS, FTP, SMTP, POP3, and IMAP.

4. User Interface Design

- Focuses on creating intuitive and user-friendly interfaces that enhance user experience and accessibility.

5. Security and Privacy

- Ensures that applications are secure from unauthorized access and data breaches, protecting user data and privacy.

6. Performance and Scalability

- Applications should be designed to perform efficiently under various loads and be scalable to handle increasing amounts of data and users.

7. Maintainability and Flexibility

- Applications should be easy to maintain and update, with a modular design that allows for flexibility and future enhancements.

These principles form the backbone of modern computer applications, ensuring they are efficient, secure, and user-friendly.

2.2 web and HTTP

The Web

The World Wide Web (WWW) is a system of interlinked hypertext documents accessed via the Internet. These documents are written in HTML (HyperText Markup Language) and can include text, images, videos, and other multimedia. Users can navigate between these documents using hyperlinks.

HTTP (Hypertext Transfer Protocol)

HTTP is the protocol used for transferring data on the Web. It defines how messages are formatted and transmitted, and how web servers and browsers should respond to various commands. Here are some key points about HTTP:

1. **Client-Server Model:** HTTP operates on a client-server model. The client (usually a web browser) sends an HTTP request to the server, which then processes the request and sends back an HTTP response.

2. **Request-Response Cycle:** The basic flow involves the client sending a request (e.g., for a web page), and the server responding with the requested resource (e.g., the HTML file of the web page).
3. **HTTP Methods:** There are several methods used in HTTP, including:
 - **GET:** Requests data from a specified resource.
 - **POST:** Submits data to be processed to a specified resource.
 - **PUT:** Updates or creates a resource.
 - **DELETE:** Deletes a specified resource.
4. **Stateless Protocol:** HTTP is stateless, meaning each request from a client to server is independent; the server does not retain any state information between requests.
5. **Versions:** HTTP has evolved over time:
 - **HTTP/1.0:** Introduced in 1996.
 - **HTTP/1.1:** Introduced in 1997, with persistent connections and chunked transfer encoding.
 - **HTTP/2:** Introduced in 2015, with multiplexing and header compression.
 - **HTTP/3:** Based on the QUIC protocol, designed to improve performance.

How It Works

When you type a URL into your browser, here's what happens:

1. **DNS Lookup:** The browser contacts a DNS server to resolve the domain name into an IP address.
2. **TCP Connection:** The browser establishes a TCP connection with the server.
3. **HTTP Request:** The browser sends an HTTP request to the server.
4. **Server Response:** The server processes the request and sends back an HTTP response, which includes the requested resource.
5. **Rendering:** The browser renders the resource (e.g., displays the web page)

2.3 E-mail

E-mail, or electronic mail, is a method of exchanging digital messages over a network, typically the Internet. It allows users to send and receive text, graphics, sounds, and even animated images.

1. **Protocols:** E-mail relies on several protocols within the TCP/IP suite. The Simple Mail Transfer Protocol (SMTP) is used to send messages, while protocols like IMAP (Internet Message Access Protocol) and POP (Post Office Protocol) are used to retrieve messages from a mail server.
2. **E-mail Servers:** These servers handle the sending, receiving, and storage of e-mail messages. When you send an e-mail, it is transferred from your e-mail client to an outgoing mail server using SMTP. The server then forwards the message to the recipient's mail server, where it can be retrieved using IMAP or POP.
3. **Components:** An e-mail message typically includes a header (with information like the sender, recipient, subject, and date) and a body (the main content of the message). Attachments can also be included, allowing users to send files along with their messages.
4. **Security:** E-mail security is crucial, as it involves protecting the content of messages from unauthorized access. This can include encryption, authentication protocols, and secure connections (like SSL/TLS) to ensure that messages are transmitted securely.

2.4 DNS in computer network

The Domain Name System (DNS) is an important component of the internet infrastructure. It acts like the internet's phonebook, translating human-friendly domain names (like `www.example.com`) into numerical IP addresses (like `192.0.2.1`) that computers use to identify each other on the network¹².

How DNS Works

When you type a web address into your browser, the DNS resolution process involves several steps:

1. **DNS Query:** Your computer sends a query to a DNS resolver, usually provided by your Internet Service Provider (ISP).
2. **Recursive Resolver:** The resolver checks its cache to see if it already has the IP address for the domain. If not, it queries other DNS servers.
3. **Root Nameserver:** The resolver first contacts a root nameserver, which directs it to the appropriate Top-Level Domain (TLD) nameserver (e.g., .com, .org).
4. **TLD Nameserver:** The TLD nameserver then directs the resolver to the authoritative nameserver for the specific domain.
5. **Authoritative Nameserver:** This server provides the IP address associated with the domain name.
6. **Response:** The resolver returns the IP address to your computer, which can then access the website.

Components of DNS

- **Domain Names:** Human-readable names like `www.example.com`.
- **IP Addresses:** Numerical addresses like `192.0.2.1`.
- **DNS Servers:** Include recursive resolvers, root nameservers, TLD nameservers, and authoritative nameservers.

Types of DNS Records

- **A Record:** Maps a domain to an IPv4 address.
- **AAAA Record:** Maps a domain to an IPv6 address.
- **CNAME Record:** Alias of one name to another.
- **MX Record:** Mail exchange servers for email routing.
- **TXT Record:** Text information for various purposes, often for verification.

Importance of DNS

- **User Convenience:** Allows users to use easy-to-remember domain names instead of complex IP addresses.
- **Scalability:** Supports the vast number of devices and services on the internet.
- **Redundancy:** Multiple DNS servers ensure reliability and availability.

DNS is fundamental to the functioning of the internet, ensuring that users can easily access websites and services.

2.5 Socket programming with TCP and UDP

Socket programming is a way to enable communication between two nodes on a network. It can be done using two main protocols: TCP (Transmission Control Protocol) and UDP (User Datagram Protocol). Here's a brief overview of both:

TCP Socket Programming

TCP is a connection-oriented protocol, which means a connection is established before data can be sent. It ensures reliable and ordered delivery of data.

Steps for TCP Socket Programming

1. Server Side:
 - Create a socket: Use the `socket()` function.
 - Bind the socket: Associate the socket with an IP address and port using the `bind()` function.
 - Listen for connections: Use the `listen()` function to wait for client connections.
 - Accept a connection: Use the `accept()` function to establish a connection with a client.
 - Send/Receive data: Use `send()` and `recv()` functions to communicate.
 - Close the connection: Use the `close()` function.
2. Client Side:
 - Create a socket: Use the `socket()` function.
 - Connect to the server: Use the `connect()` function to establish a connection.
 - Send/Receive data: Use `send()` and `recv()` functions to communicate.
 - Close the connection: Use the `close()` function.

UDP Socket Programming

UDP is a connectionless protocol, which means data can be sent without establishing a connection. It is faster but does not guarantee reliable or ordered delivery.

Steps for UDP Socket Programming

1. Server Side:
 - Create a socket: Use the `socket()` function.
 - Bind the socket: Associate the socket with an IP address and port using the `bind()` function.
 - Receive data: Use the `recvfrom()` function to receive data.
 - Send data: Use the `sendto()` function to send data.
 - Close the socket: Use the `close()` function.

2. Client Side:

- Create a socket: Use the `socket()` function.
- Send data: Use the `sendto()` function to send data.
- Receive data: Use the `recvfrom()` function to receive data.
- Close the socket: Use the `close()` function.

Key Differences

- Connection: TCP requires a connection to be established, while UDP does not.
- Reliability: TCP ensures reliable delivery, while UDP does not.
- Speed: UDP is generally faster due to the lack of connection overhead.

Socket programming is essential for creating networked applications, such as web servers, chat applications, and more.

TCP socket programming in C, demonstrating both the server and client sides.

Server Side

1. **Create a socket:** Use the `socket()` function.
2. **Bind the socket:** Associate the socket with an IP address and port using the `bind()` function.
3. **Listen for connections:** Use the `listen()` function to wait for client connections.
4. **Accept a connection:** Use the `accept()` function to establish a connection with a client.
5. **Send/Receive data:** Use `send()` and `recv()` functions to communicate.
6. **Close the connection:** Use the `close()` function.

Server Code Example

```
#include <stdio.h>
#include <netdb.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>
```

```

#define MAX 80
#define PORT 8080
#define SA struct sockaddr

void func(int connfd) {
    char buff[MAX];
    int n;
    for (;;) {
        bzero(buff, MAX);
        read(connfd, buff, sizeof(buff));
        printf("From client: %s\t To client : ", buff);
        bzero(buff, MAX);
        n = 0;
        while ((buff[n++] = getchar()) != '\n');
        write(connfd, buff, sizeof(buff));
        if (strncmp("exit", buff, 4) == 0) {
            printf("Server Exit...\n");
            break;
        }
    }
}

int main() {
    int sockfd, connfd, len;
    struct sockaddr_in servaddr, cli;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == -1) {
        printf("Socket creation failed...\n");
        exit(0);
    } else {
        printf("Socket successfully created..\n");
    }
    bzero(&servaddr, sizeof(servaddr));

    servaddr.sin_family = AF_INET;

```

```

servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
servaddr.sin_port = htons(PORT);

if ((bind(sockfd, (SA*)&servaddr, sizeof(servaddr))) != 0) {
    printf("Socket bind failed...\n");
    exit(0);
} else {
    printf("Socket successfully binded..\n");
}

if ((listen(sockfd, 5)) != 0) {
    printf("Listen failed...\n");
    exit(0);
} else {
    printf("Server listening..\n");
}
len = sizeof(cli);

connfd = accept(sockfd, (SA*)&cli, &len);
if (connfd < 0) {
    printf("Server accept failed...\n");
    exit(0);
} else {
    printf("Server accepted the client...\n");
}

func(connfd);
close(sockfd);
}

```

Client Side

1. **Create a socket:** Use the `socket()` function.
2. **Connect to the server:** Use the `connect()` function to establish a connection.
3. **Send/Receive data:** Use `send()` and `recv()` functions to communicate.
4. **Close the connection:** Use the `close()` function.

Client Code Example

```
#include <stdio.h>
#include <netdb.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>

#define MAX 80
#define PORT 8080
#define SA struct sockaddr

void func(int sockfd) {
    char buff[MAX];
    int n;
    for (;;) {
        bzero(buff, sizeof(buff));
        printf("Enter the string : ");
        n = 0;
        while ((buff[n++] = getchar()) != '\n');
        write(sockfd, buff, sizeof(buff));
        bzero(buff, sizeof(buff));
        read(sockfd, buff, sizeof(buff));
        printf("From Server : %s", buff);
        if ((strcmp(buff, "exit", 4)) == 0) {
            printf("Client Exit...\n");
            break;
        }
    }
}

int main() {
    int sockfd, connfd;
    struct sockaddr_in servaddr, cli;
```

```

sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd == -1) {
    printf("Socket creation failed...\n");
    exit(0);
} else {
    printf("Socket successfully created..\n");
}
bzero(&servaddr, sizeof(servaddr));

servaddr.sin_family = AF_INET;
servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");
servaddr.sin_port = htons(PORT);

if (connect(sockfd, (SA*)&servaddr, sizeof(servaddr)) != 0) {
    printf("Connection with the server failed...\n");
    exit(0);
} else {
    printf("Connected to the server..\n");
}

func(sockfd);
close(sockfd);
}

```

This example demonstrates a simple chat application where the client sends a message to the server, and the server echoes it back. The connection is terminated when the client sends the message "exit".



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

Faculty of Engineering (Degree)
Semester – 5th
Subject Name –Computer Networks
Subject Code - BE103404
Faculty Name – Radhika Gokani
Chapter -3

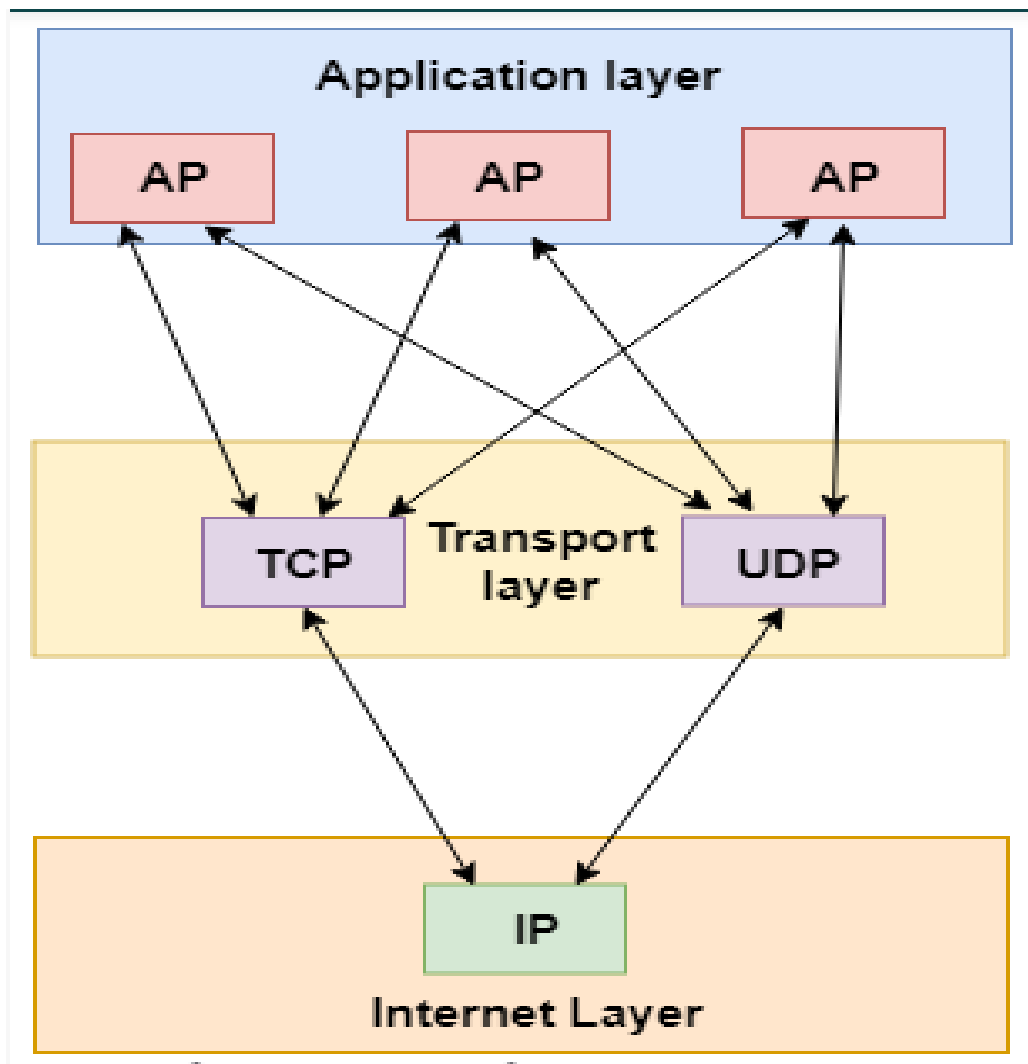
Transport Layer

3 & 3.1 introduction and Transport layer services

The **transport layer** is a crucial component of computer networks, specifically in the OSI (Open Systems Interconnection) model, where it is the fourth layer

Functions of the Transport Layer

1. **End-to-End Communication:** The transport layer ensures reliable data transfer between host systems. It establishes a connection, maintains it during data transfer, and terminates it once the communication is complete.
2. **Segmentation and Reassembly:** It breaks down large data from the application layer into smaller segments for transmission. At the destination, these segments are reassembled into the original data stream.
3. **Flow Control:** This function prevents network congestion by controlling the rate of data transmission. Protocols like the sliding window protocol are used to manage this.
4. **Error Detection and Correction:** The transport layer uses checksums to detect errors in the transmitted data. If errors are found, it requests retransmission of the corrupted segments.
5. **Multiplexing and Demultiplexing:** It allows multiple applications to use the network simultaneously by assigning unique identifiers to each data stream, ensuring that data is delivered to the correct application.



Common Transport Layer Protocols

- **TCP (Transmission Control Protocol):** Provides reliable, ordered, and error-checked delivery of data.

The transport layer plays a vital role in ensuring that data is transmitted accurately and efficiently across networks, making it an essential part of modern communication systems.

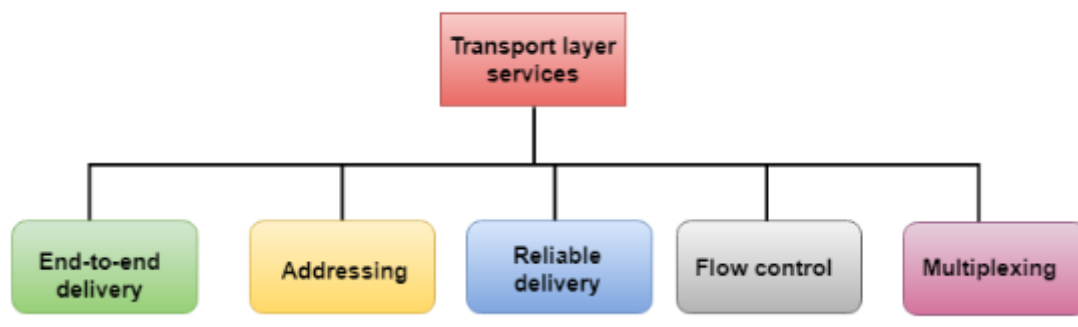
- **UDP (User Datagram Protocol):** Offers a simpler, connectionless service with minimal error recovery, suitable for applications needing fast, efficient transmission
 - The transport layer is a 4th layer from the top.
 - The main role of the transport layer is to provide the communication services directly to the application processes running on different hosts.
 - The transport layer provides a logical communication between application processes running on different hosts. Although the application processes on

different hosts are not physically connected, application processes use the logical communication provided by the transport layer to send the messages to each other.

- The transport layer protocols are implemented in the end systems but not in the network routers.
- A computer network provides more than one protocol to the network applications. For example, TCP and UDP are two transport layer protocols that provide a different set of services to the network layer.
- All transport layer protocols provide multiplexing/demultiplexing service. It also provides other services such as reliable data transfer, bandwidth guarantees, and delay guarantees.
- Each of the applications in the application layer has the ability to send a message by using TCP or UDP. The application communicates by using either of these two protocols. Both TCP and UDP will then communicate with the internet protocol in the internet layer. The applications can read and write to the transport layer. Therefore, we can say that communication is a two-way process.

Services provided by the Transport Layer

The services provided by the transport layer are similar to those of the data link layer. The data link layer provides the services within a single network while the transport layer provides the services across an internetwork made up of many networks. The data link layer controls the physical layer while the transport layer controls all the lower layers.



The services provided by the transport layer protocols can be divided into five categories:

- End-to-end delivery
 - Addressing
 - Reliable delivery
 - Flow control
 - Multiplexing
-

3.2 Multiplexing and Demultiplexing

Multiplexing and **demultiplexing** are essential processes in computer networks, particularly at the transport layer.

Multiplexing/demultiplexing

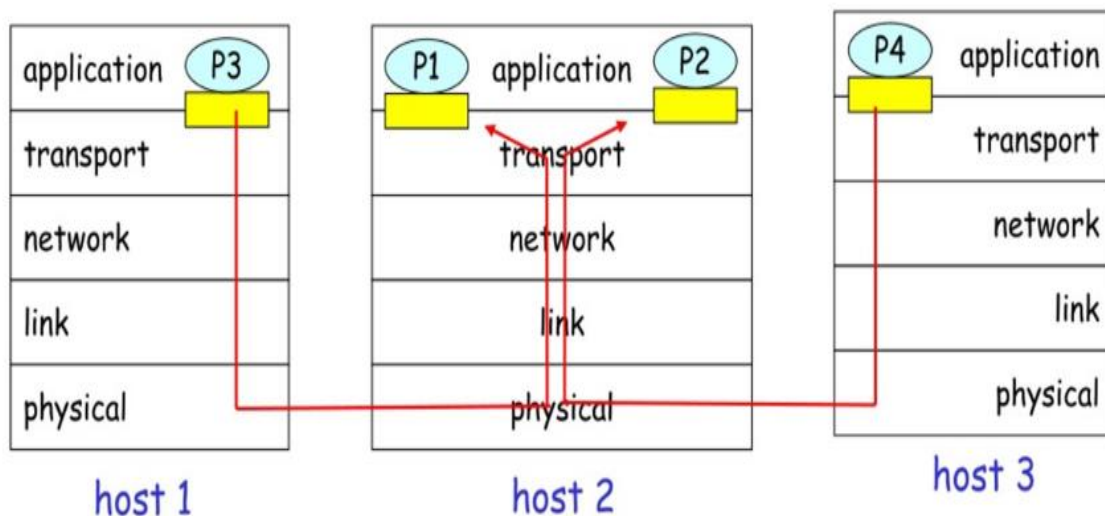
Demultiplexing at rcv host:

delivering received segments to correct socket

Multiplexing at send host:

gathering data from multiple sockets, enveloping data with header (later used for demultiplexing)

■ = socket ○ = process



Multiplexing

Multiplexing involves combining multiple data streams from different applications or sources into a single, unified data stream for transmission over a network. This process is crucial for efficient use of network resources. There are several types of multiplexing:

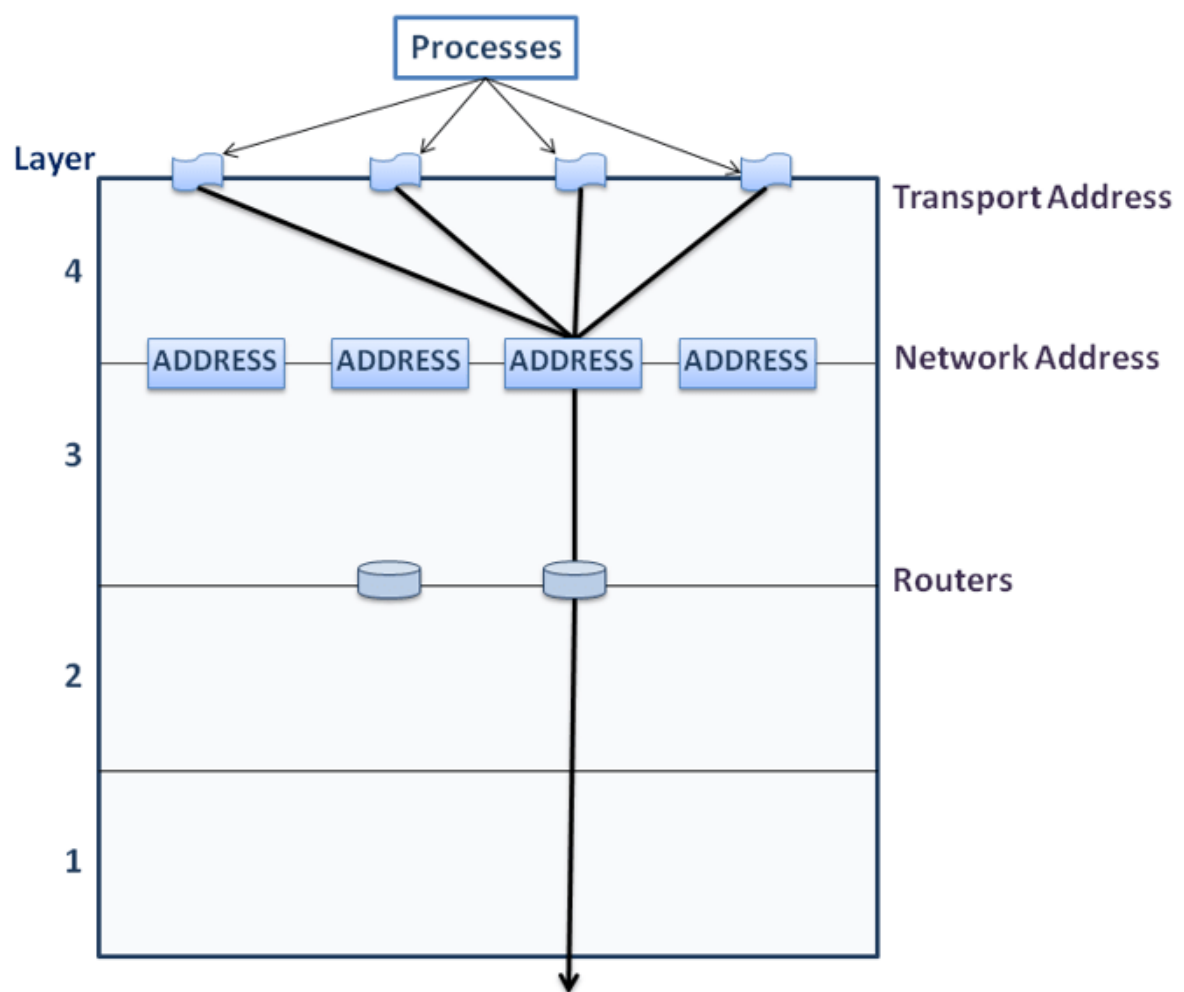
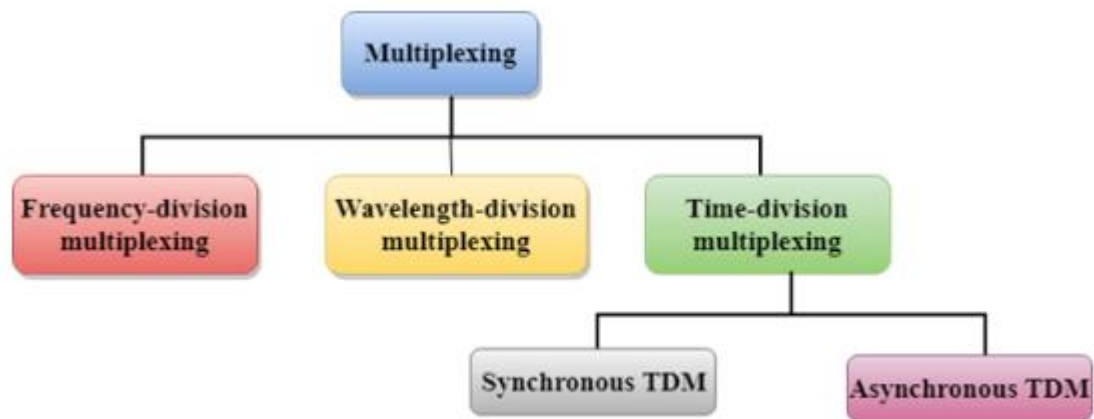


Figure 1: Multiplexing Mechanism

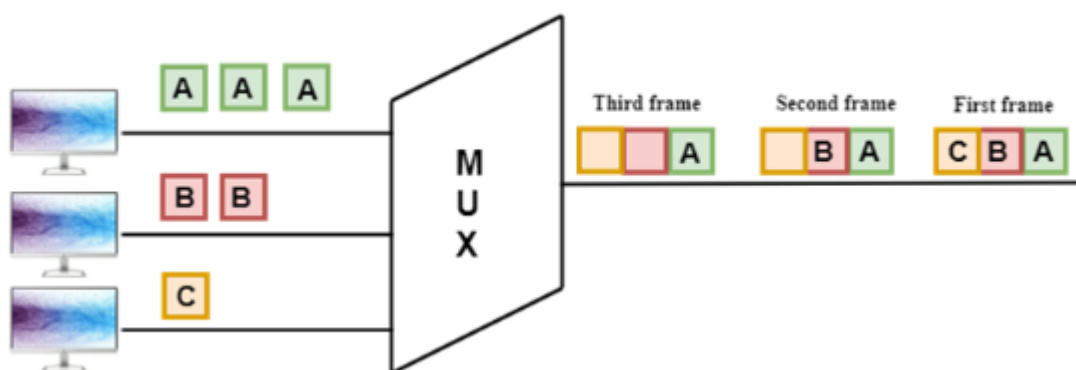
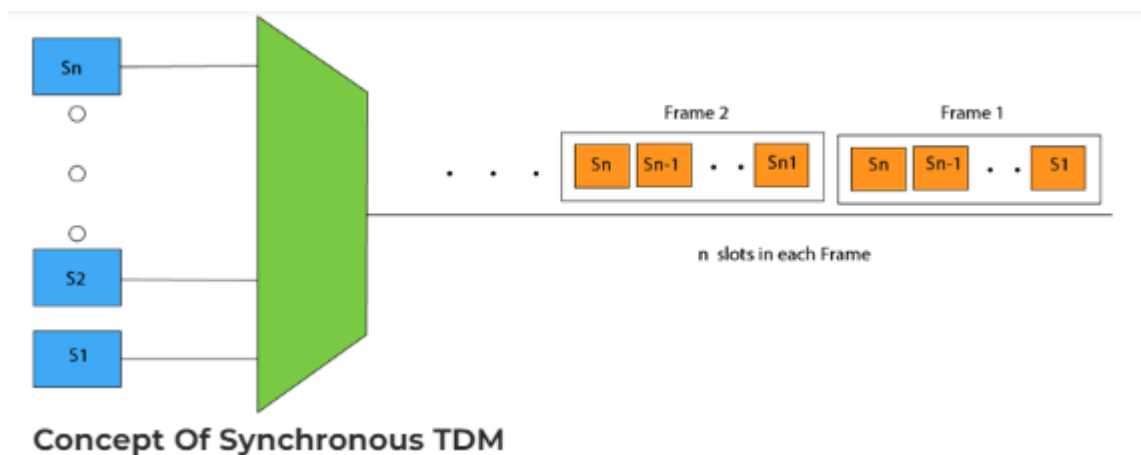


1. **Time-Division Multiplexing (TDM):** Divides the available bandwidth into time slots, with each data stream assigned a specific time slot.

- It is a digital technique.
- In Frequency Division Multiplexing Technique, all signals operate at the same time with different frequency, but in case of Time Division Multiplexing technique, all signals operate at the same frequency with different time.
- In **Time Division Multiplexing technique**, the total time available in the channel is distributed among different users. Therefore, each user is allocated with different time interval known as a Time slot at which data is to be transmitted by the sender.
- A user takes control of the channel for a fixed amount of time.
- In Time Division Multiplexing technique, data is not transmitted simultaneously rather the data is transmitted one-by-one.
- In TDM, the signal is transmitted in the form of frames. Frames contain a cycle of time slots in which each frame contains one or more slots dedicated to each user.
- It can be used to multiplex both digital and analog signals but mainly used to multiplex digital signals.

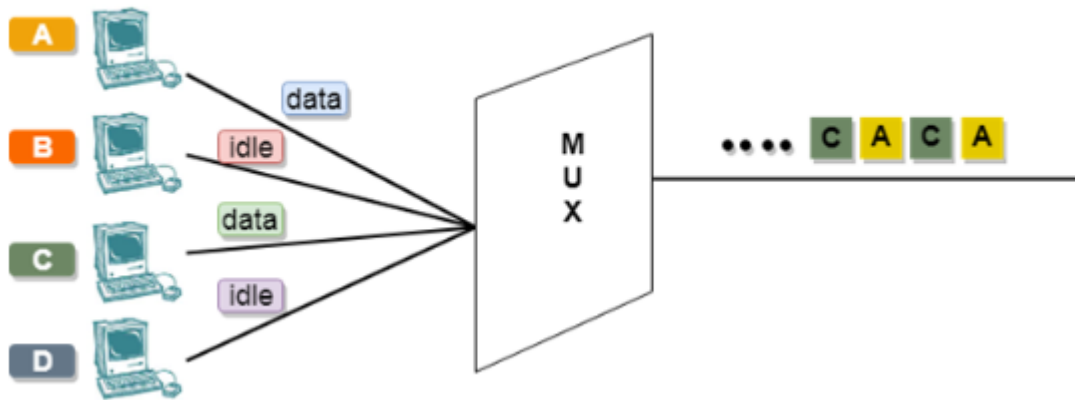
There are two types of TDM:

- **Synchronous TDM**

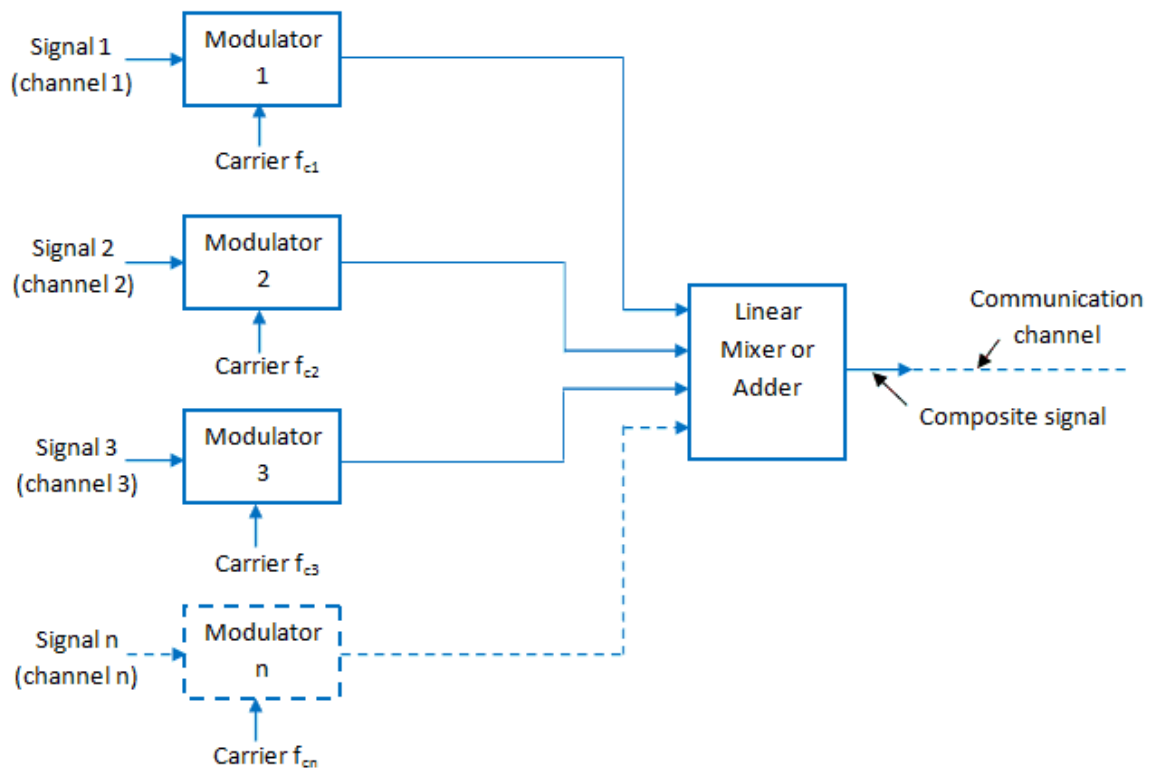


Asynchronous TDM

Concept Of Asynchronous TDM



2. **Frequency-Division Multiplexing (FDM):** Allocates different frequency bands to different data streams.



Frequency-Division Multiplexing

Frequency-Division Multiplexing (FDM) is a technique used in telecommunications to transmit multiple signals simultaneously over a single communication medium. Here's a brief overview:

How FDM Works

- **Bandwidth Division:** The total bandwidth of the communication medium is divided into a series of non-overlapping frequency bands. Each band carries a separate signal.
- **Carrier Signals:** Each signal is modulated onto a different carrier frequency. This means that each signal is transmitted at a unique frequency, preventing interference between signals.
- **Multiplexing and Demultiplexing:** At the transmitting end, signals are combined (multiplexed) and sent over the medium. At the receiving end, they are separated (demultiplexed) back into their original signals.

Applications of FDM

- **Radio and TV Broadcasting:** Multiple radio and TV channels are transmitted simultaneously over the airwaves.
- **Cable Television:** Many TV channels are carried on a single cable.
- **Telephone Systems:** Multiple telephone calls are transmitted through high-capacity trunk lines.
- **DSL Modems:** Large amounts of computer data are transmitted through twisted pair telephone lines.

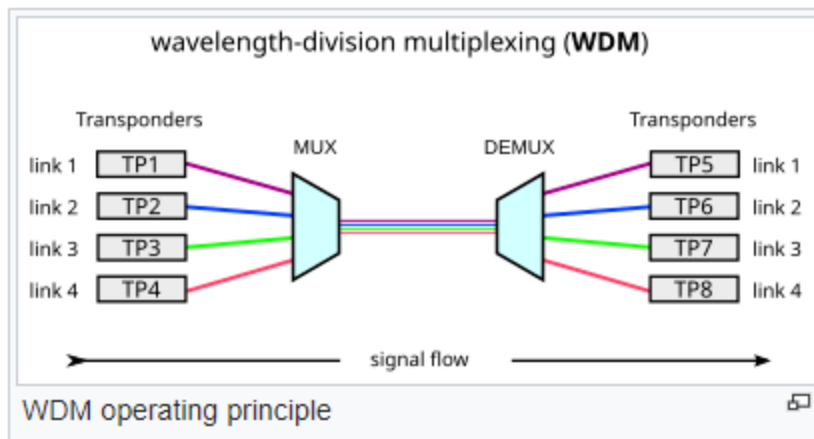
Advantages

- **Efficient Use of Bandwidth:** Multiple signals can be transmitted over a single channel.
- **No Time Synchronization Required:** Easier to implement as it doesn't require precise time synchronization.
- **Low Implementation Cost:** Relatively simple and inexpensive to implement.

Disadvantages

- **Limited Capacity:** The number of signals that can be transmitted is limited by the available bandwidth.

-
-
3. **Wavelength-Division Multiplexing (WDM):** Used in fiber-optic communications, where different wavelengths (colors) of light carry different data streams.



Wavelength-Division Multiplexing (WDM) is a technology used in fiber-optic communications to increase bandwidth by allowing multiple data streams to be transmitted simultaneously over a single optical fiber. This is achieved by using different wavelengths (or colors) of laser light for each data stream.

There are two main types of WDM:

1. **Coarse Wavelength-Division Multiplexing (CWDM):** This type uses fewer wavelengths (typically less than 8) and is suitable for short-range communications. It is cost-effective but less efficient in terms of spectral usage.

2. **Dense Wavelength-Division Multiplexing (DWDM):** This type uses many more wavelengths (often 40 or more) and is used for long-distance and high-capacity applications. It is more expensive but allows for much higher data transmission rates.

WDM systems use a multiplexer at the transmitter to combine the different wavelengths and a demultiplexer at the receiver to separate them. This technology allows telecommunications companies to expand their network capacity without laying additional fiber.

Demultiplexing

Demultiplexing is the reverse process of multiplexing. At the receiving end, the combined data stream is separated back into its original individual data streams and delivered to the appropriate applications. This ensures that data is correctly routed to its intended destination.

Demultiplexing in computer networking is a crucial process that occurs at the transport layer of the OSI model. It involves directing incoming data packets to the correct application process based on information in the packet headers, such as port numbers.

1. **Transport Layer Role:** The transport layer protocols, such as TCP and UDP, handle demultiplexing. They use port numbers to identify the receiving application. For instance, TCP relies on a combination of IP addresses and port numbers, while UDP uses only port numbers.
2. **Process:** When data packets arrive at the destination, the transport layer examines the destination port number in the packet header. This port number helps the transport layer determine which application should receive the data. The process of separating these packets and delivering them to the appropriate application is called demultiplexing.
3. **Types of Demultiplexing:**
 - **Connectionless Demultiplexing:** Used by UDP, where each packet is treated independently, and the destination port number is used to direct the packet to the correct application.
 - **Connection-Oriented Demultiplexing:** Used by TCP, where a connection is established between the sender and receiver before data transfer. The connection is identified by a combination of source and destination IP addresses and port numbers.
4. **Example:** Imagine you are using multiple applications on your device, like a web browser and an email client. When data packets arrive, demultiplexing ensures that web data goes to the browser and email data goes to the email client, even though both types of data are coming through the same network connection.

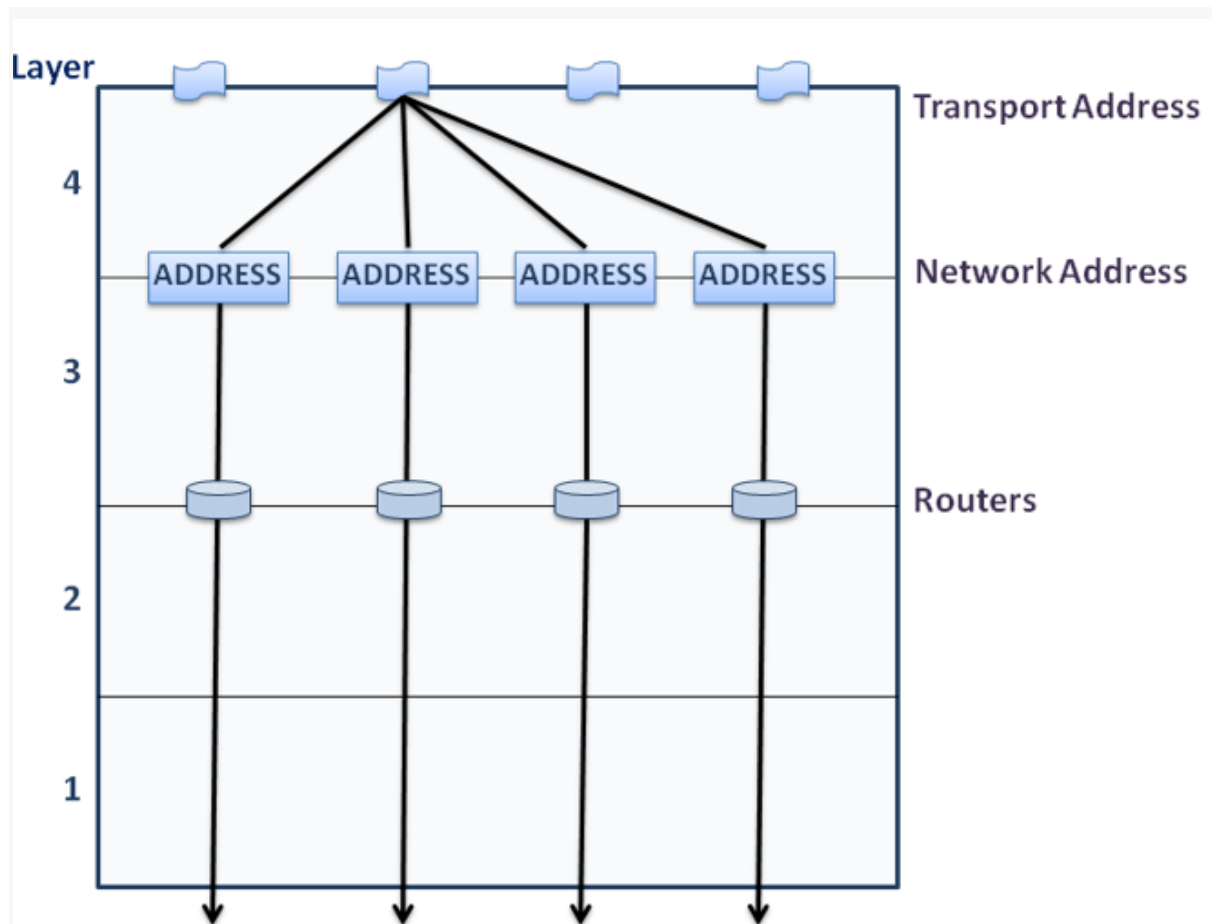


Figure 2: De-Multiplexing Mechanism



How It Works in the Transport Layer

In the transport layer, protocols like TCP and UDP handle multiplexing and demultiplexing using port numbers:

- **Multiplexing:** The transport layer collects data from multiple applications, adds a header containing source and destination port numbers, and sends it as a single stream.
- **Demultiplexing:** At the receiving end, the transport layer examines the port numbers in the header to determine which application should receive the data.

Practical Example

Imagine you are using multiple applications on your computer, such as a web browser and an email client. When data is sent from these applications, multiplexing combines the data into a single stream. Upon reaching the destination, demultiplexing ensures that the web browser data goes to the web browser and the email data goes to the email client.

These processes are fundamental for efficient and accurate data transmission in modern networks.

3.3 Connectionless transport (UDP)

The **User Datagram Protocol (UDP)** is a connectionless transport layer protocol used in computer networks. Unlike the Transmission Control Protocol (TCP), UDP does not establish a connection before sending data, making it faster but less reliable. Here are some key points about UDP:

1. **Connectionless Nature:** UDP sends data packets, called datagrams, without establishing a connection between the sender and receiver. Each packet is sent independently, and they may take different paths to reach the destination.
2. **No Guarantee of Delivery:** UDP does not guarantee the delivery, order, or integrity of the data packets. This means packets can be lost, duplicated, or received out of order.
3. **Low Overhead:** The protocol has minimal overhead, with a simple header structure of only 8 bytes, making it efficient for applications that require fast transmission and can tolerate some data loss.
4. **Applications:** UDP is commonly used in applications where speed is critical and occasional data loss is acceptable.

Examples include:

- **Streaming media** (audio and video)
- **Online gaming**
- **Voice over IP (VoIP)**
- **Domain Name System (DNS) queries**

5. **Port Numbers:** UDP uses port numbers to distinguish different user requests or processes. Each datagram includes source and destination port numbers

3.4 Principles of reliable data transfer

Reliable data transfer in computer networks ensures that data is transmitted accurately and in the correct order, despite potential issues like data loss, corruption, or duplication.

Principles:

1. **Error Detection and Correction:** Mechanisms like checksums and cyclic redundancy checks (CRC) are used to detect errors in transmitted data. If errors are detected, protocols like TCP can request retransmission of the corrupted data.
2. **Acknowledgments (ACKs) and Negative Acknowledgments (NAKs):** The receiver sends an ACK to the sender to confirm successful receipt of data. If data is corrupted or lost, a NAK is sent, prompting the sender to retransmit the data.
3. **Sequence Numbers:** Each packet of data is assigned a unique sequence number. This helps in reordering packets that may arrive out of order and in detecting duplicate packets.
4. **Flow Control:** This ensures that the sender does not overwhelm the receiver with too much data at once. Protocols like TCP use a sliding window mechanism to manage the flow of data.
5. **Congestion Control:** This prevents network congestion by adjusting the rate of data transmission based on network conditions. TCP uses algorithms like slow start and congestion avoidance to manage congestion.
6. **Timeouts and Retransmissions:** If an acknowledgment is not received within a certain time frame, the sender assumes the packet was lost and retransmits it.

These principles are implemented in various layers of the network protocol stack, with TCP being a prime example of a protocol that ensures reliable data transfer over an unreliable network layer.

3.5 Connection-oriented transport (TCP)

Transmission Control Protocol (TCP) is a fundamental protocol in computer networks, specifically at the transport layer of the OSI model. Here are some key points about TCP:

1. **Connection-Oriented:** TCP is a connection-oriented protocol, meaning it establishes a connection between the sender and receiver before data transmission begins. This ensures that data packets are delivered in the correct order and without errors.
2. **Reliable Data Transfer:** TCP provides reliable data transfer by using acknowledgments and retransmissions. If a packet is lost or corrupted during transmission, TCP will retransmit it.
3. **Flow Control:** TCP uses flow control mechanisms to prevent network congestion. It adjusts the rate of data transmission based on the receiver's ability to process the incoming data.
4. **Error Detection and Correction:** TCP includes error-checking mechanisms to detect any errors in the transmitted data. If errors are found, the data is retransmitted.
5. **Segmentation and Reassembly:** TCP breaks down large data messages into smaller segments for transmission. These segments are reassembled into the original message at the receiving end.
6. **Applications:** TCP is widely used in applications where reliable data transfer is crucial, such as web browsing (HTTP/HTTPS), email (SMTP), and file transfer (FTP)

Types of Connection-Oriented Service :

Service	Example
Reliable Message Stream	Sequence of pages, etc.
Reliable Byte Stream	Song Download, etc.
Unreliable Connection	VoIP (Voice Over Internet Protocol)

3.6 Congestion control

Congestion control in computer networks is essential for maintaining smooth and efficient data flow. When too much data is sent through the network at once, it can lead to delays and data loss, similar to traffic congestion on a busy road. Here are some key points about congestion control:

What is Congestion?

Congestion occurs when the network is overloaded with data, causing delays and potential data loss. This can significantly impact the performance and reliability of the network.

Effects of Congestion Control

1. **Improved Network Stability:** Helps prevent network overload, ensuring stable and reliable performance.
2. **Reduced Latency and Packet Loss:** Manages traffic to minimize delays and data loss.
3. **Enhanced Throughput:** Optimizes resource usage, allowing more data to be transmitted efficiently.
4. **Fairness in Resource Allocation:** Ensures equitable bandwidth distribution among users.
5. **Better User Experience:** Smooth data flow leads to faster and more reliable applications and services.

6. **Mitigation of Network Congestion Collapse:** Prevents severe congestion that can make the network almost unusable.

Congestion Control Algorithms

1. **Leaky Bucket Algorithm:** Used for traffic shaping or rate-limiting, ensuring data is sent at a consistent rate.
2. **Token Bucket Algorithm:** Allows bursty data transmission while controlling the average data rate.

Congestion Control Techniques

1. **Open Loop Congestion Control:** Prevents congestion before it happens through policies like retransmission, window, discarding, acknowledgment, and admission policies.
2. **Closed Loop Congestion Control:** Addresses congestion after it occurs using techniques like backpressure, where a congested node stops receiving packets from upstream nodes

3.7 TCP congestion control

TCP (Transmission Control Protocol) congestion control is a critical mechanism used to manage data flow and prevent network congestion. Here's a brief overview of how it works:

Phases of TCP Congestion Control

1. **Slow Start Phase:**
 - **Exponential Growth:** Initially, the congestion window (CWND) size increases exponentially. For example, if the initial CWND is 1 segment, it doubles after each round-trip time (RTT) as long as acknowledgments are received successfully.
2. **Congestion Avoidance Phase:**
 - **Additive Increase:** Once the CWND reaches a threshold (ssthresh), it grows linearly. For instance, if the CWND is 20 segments, it increases by 1 segment per RTT.
3. **Congestion Detection Phase:**
 - **Multiplicative Decrease:** When congestion is detected (e.g., through packet loss or timeout), the CWND is reduced. If a timeout occurs, the CWND is set to 1, and the ssthresh is halved. If three duplicate ACKs are received, the CWND is set to the ssthresh value.

Algorithms and Techniques

- **Additive Increase/Multiplicative Decrease (AIMD):** This algorithm combines linear growth of the CWND with an exponential reduction when congestion occurs, ensuring fair bandwidth distribution among multiple flows.

- **Congestion Window (CWND):** This is a dynamic value maintained by the sender to control the number of unacknowledged packets in transit, preventing network overload.

Importance

TCP congestion control ensures efficient and stable data transmission across networks, adapting to current network conditions to avoid congestion collapse and maintain reliable communication



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

Faculty of Engineering (Degree)
Semester – 5th
Subject Name –Computer Networks
Subject Code - BE103404
Faculty Name – Radhika Gokani
Chapter - 4

4.1 Network Layer

The Network Layer is an essential part of the OSI (Open Systems Interconnection) model. It's the third layer, positioned between the Data Link Layer (Layer 2) and the Transport Layer (Layer 4).

Functions of the Network Layer

- **Routing:** Determines the optimal path for data to travel from the source to the destination. Routers operate at this layer.
- **Logical Addressing:** Assigns IP addresses to devices, ensuring each device can be uniquely identified on the network. This is crucial for data delivery to the correct destination.
- **Packet Forwarding:** Moves packets from one network segment to another, ensuring they reach their final destination.
- **Fragmentation and Reassembly:** Breaks down large packets into smaller fragments to pass through networks with smaller maximum transmission units (MTUs) and reassembles them at the destination.

Concepts

- **IP Addressing:** The Internet Protocol (IP) is used for addressing and routing packets. IPv4 (32-bit addresses) and IPv6 (128-bit addresses) are the most common protocols.
- **Routing Algorithms:** Used to determine the best path for data to travel across a network. Examples include OSPF (Open Shortest Path First) and BGP (Border Gateway Protocol).
- **Packet Switching:** Involves breaking down data into packets that are transmitted over various routes and reassembled at the destination.

Importance of the Network Layer

- **Scalability:** Allows the network to grow and support more devices without significant changes to the underlying infrastructure.
- **Interconnectivity:** Ensures different network types (LAN, WAN, MAN) can connect and communicate efficiently.
- **Efficiency:** Optimizes the use of network resources through routing and addressing.

4.1 Introduction to forwarding and routing

Forwarding and routing are fundamental functions of the network layer in computer networking.

Forwarding

Forwarding refers to the process of transferring a packet from an incoming interface to the appropriate outgoing interface within a single router. Think of it like a mail sorting system within a post office, where the packet is moved to the next hop on its journey toward its final destination.

Tasks in forwarding includes

- **Lookup and Forwarding:** The router examines the packet's destination IP address and consults its forwarding table to determine the next hop.
- **Switching Fabric:** Internal processes move the packet from the input port to the correct output port.
- **Queuing:** If there is congestion, packets may be queued until the outgoing link is available.

Routing

Routing, on the other hand, is the overall process of determining the paths that packets take from source to destination across multiple routers. It's akin to planning the route on a map. Routing involves:

- **Routing Algorithms:** Algorithms like Distance Vector, Link State, and Path Vector calculate the optimal path for packet delivery.
- **Routing Tables:** Routers build and maintain these tables to know where to forward packets. These tables are updated dynamically based on network topology changes.
- **Routing Protocols:** Protocols such as OSPF (Open Shortest Path First), BGP (Border Gateway Protocol), and EIGRP (Enhanced Interior Gateway Routing Protocol) are used to disseminate routing information between routers.

Differences

1. **Scope:** Forwarding is a local activity within a router; routing is a global process involving the entire network.
- 2.

- 3.
4. **Function:** Forwarding deals with moving packets to their next hop, while routing determines the entire path from source to destination.
5. **Process:** Forwarding is implemented in hardware for speed, while routing is often managed by software to allow for complex computations.

By combining these two functions, networks efficiently deliver packets from source to destination even across vast distances and complex topologies.

4.2 Network Service models

Network service models define the characteristics of end-to-end data transport between two points in a network, typically between a sender and a receiver.

- **Circuit-Switched Network:** Establishes a dedicated physical path between sender and receiver for the duration of the connection. Traditional telephone networks are an example.
- **Packet-Switched Network:** Data is divided into packets, which are routed individually over various paths in the network. The Internet is a prime example of a packet-switched network.
- **Message-Switched Network:** Similar to packet switching, but entire messages are routed one hop at a time. Each switch stores the entire message before forwarding it.
- **Virtual Circuit Network:** Combines elements of both circuit and packet switching. A virtual path is established before data transfer, but data is still sent in packets.
- **Datagram Network:** Each packet (datagram) is treated independently, with no pre-established path. Each packet may take a different route to the destination.

4.3 Virtual and Datagram networks

Virtual Networks

Imagine a network diagram with nodes (representing devices) connected by lines (representing paths). In a virtual network, you'd see dedicated paths or circuits between nodes, clearly marked and consistently connecting the same nodes throughout the communication session.

- **Concept:** Similar to how you book a single path on a scenic route, a virtual network reserves a defined path for data packets between sender and receiver. This path, known as a virtual circuit, remains consistent throughout the communication session.
- **Connection-Oriented:** It establishes a connection before any data transfer begins and maintains it until the end.

- **Data Delivery:** Packets arrive in the sequence they were sent, making it reliable for applications like video streaming or VoIP where order and timing are crucial.
- **Examples:** ATM (Asynchronous Transfer Mode), Frame Relay.

Datagram Network

In contrast, a datagram network diagram would show nodes connected by multiple possible paths, with data packets moving independently along various routes. The lines wouldn't indicate a fixed path but rather a flexible network where packets find their way dynamically.

If you need these visuals for a specific purpose, you might want to search for images online using keywords like "Virtual Network diagram" and "Datagram Network diagram."

- **Concept:** This is more like sending letters through a postal service, where each packet of data (datagram) is treated independently and can take any available route to reach the destination.
- **Connectionless:** No fixed path is established; each datagram finds its own route based on the network's current state.
- **Data Delivery:** Packets may arrive out of order, get lost, or duplicate, making it less reliable for applications that require sequence but suitable for emails or web browsing where order is less critical.
- **Examples:** IP (Internet Protocol), UDP (User Datagram Protocol).

In essence, virtual networks provide a steady, consistent path for data, while datagram networks offer flexibility and efficiency in routing. It's like choosing between a dedicated train track for your journey versus a network of highways.

4.4 study of router

Routers are fascinating devices at the heart of modern networking, directing traffic between different networks and ensuring data packets get to their destinations efficiently. Here are a few key points to understand when studying routers:

1. **Functionality:** Routers connect multiple networks together. They determine the best path for data to travel from its source to its destination using routing tables and protocols.
2. **Routing Protocols:** These protocols define how routers communicate with each other. Common examples include:
 - **RIP (Routing Information Protocol):** Simple and commonly used in smaller networks.
 - **OSPF (Open Shortest Path First):** More complex and used in larger, scalable networks.
 - **BGP (Border Gateway Protocol):** Essential for the internet, it manages how packets are routed across different autonomous systems (e.g., ISPs).
3. **Types of Routers:**
 - **Home Routers:** Usually combine a router, switch, and access point in one device, handling internet access in homes.

- **Enterprise Routers:** More powerful and support extensive networks within large organizations.
 - **Core Routers:** Used by service providers to route data across the internet backbone.
4. **Security:** Routers play a crucial role in network security, using firewalls, VPNs, and access control lists (ACLs) to protect data and control access.
 5. **Configuration:** Understanding how to configure a router, often done through a web interface or command-line interface (CLI), is key. This includes setting IP addresses, subnet masks, and default gateways.

4.5 IP protocol and addressing in the Internet

IP protocol and addressing in the Internet

The Internet Protocol (IP) is a key component of the internet, allowing different devices to communicate with each other by assigning unique addresses and directing data packets. Let's take a closer look at how IP protocol and addressing work on the internet:

IP Protocol

IP is responsible for routing and addressing data packets so they can travel across networks. There are two main versions:

1. **IPv4 (Internet Protocol version 4):**
 - **Address Format:** IPv4 addresses are 32-bit numbers, typically represented in decimal as four groups of numbers separated by periods, e.g., 192.168.1.1.
 - **Address Space:** It provides around 4.3 billion unique addresses, which are becoming increasingly scarce.
 - **Use:** Still widely used, despite the limited address space.
2. **IPv6 (Internet Protocol version 6):**
 - **Address Format:** IPv6 addresses are 128-bit numbers, written in hexadecimal and separated by colons, e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334.
 - **Address Space:** Offers an enormous number of unique addresses, vastly expanding the available address space.
 - **Use:** Gaining adoption as a solution to IPv4 exhaustion.

IP Addressing

IP addressing is the system used to assign unique identifiers to each device connected to a network. Here are key aspects:

1. **Types of IP Addresses:**
 - **Public IP Addresses:** Assigned by ISPs and are unique across the internet. They allow devices to be accessible from outside their local network.

- **Private IP Addresses:** Used within private networks and not routable on the internet. Typical ranges include 192.168.x.x, 172.16.x.x - 172.31.x.x, and 10.x.x.x.
- 2. **Static vs. Dynamic IP Addresses:**
 - **Static IP Addresses:** Manually configured and do not change. Useful for servers and network devices requiring consistent access.
 - **Dynamic IP Addresses:** Assigned by DHCP servers and can change over time. Commonly used for most client devices like smartphones, laptops, and desktops.

Subnetting

Subnetting divides a larger network into smaller, more manageable segments (subnets), improving organization and efficiency. Each subnet has a unique subnet mask that defines its network and host portions.

Routing

Routers direct data packets from one network to another using routing tables and protocols. They determine the best path for data to travel. Key routing protocols include:

- **RIP (Routing Information Protocol):** Simple and used in smaller networks.
- **OSPF (Open Shortest Path First):** More complex and used in larger networks.
- **BGP (Border Gateway Protocol):** Essential for routing data across the internet's backbone.

NAT (Network Address Translation)

NAT allows multiple devices on a local network to share a single public IP address for accessing the internet. This helps conserve public IP addresses and adds a layer of security by masking internal IP addresses.

4.6 Routing algorithms

Routing algorithms are crucial for determining the most efficient paths for data packets to travel across networks. Here are some key routing algorithms used in networking:

1. Distance Vector Routing Algorithms

These algorithms calculate the best path based on distance, typically measured in hops. Each router shares its knowledge of the entire network with its immediate neighbors.

- **RIP (Routing Information Protocol):** A well-known distance vector protocol. It uses hop count as the metric, with a maximum limit of 15 hops. It's simple but not ideal for large networks due to its limited scalability and slow convergence.

2. Link State Routing Algorithms

These algorithms build a complete map of the network and use it to determine the shortest path. They are more complex and require more resources than distance vector algorithms.

- **OSPF (Open Shortest Path First):** A widely used link state protocol. It uses the Shortest Path First (SPF) algorithm to calculate the shortest path tree. It's highly scalable and suitable for large networks.
- **IS-IS (Intermediate System to Intermediate System):** Similar to OSPF, it's used in larger networks and ISPs. It also uses the SPF algorithm and can handle large amounts of routing information.

3. Path Vector Protocols

These protocols maintain the path information that gets updated dynamically as the path changes.

- **BGP (Border Gateway Protocol):** The core routing protocol of the internet. It maintains paths to different networks and makes routing decisions based on path attributes and policies. BGP ensures that data packets are routed efficiently across the global internet.

4. Hybrid Routing Algorithms

These algorithms combine elements of both distance vector and link state algorithms.

- **EIGRP (Enhanced Interior Gateway Routing Protocol):** Developed by Cisco, it's a hybrid protocol that uses distance vectors and link state metrics. It offers fast convergence and scalability while maintaining simplicity.

Routing Metrics

Routing algorithms use different metrics to determine the best path, such as:

- **Hop Count:** The number of hops a packet takes to reach its destination.
- **Bandwidth:** The available data transfer capacity of a link.
- **Delay:** The time it takes for a packet to travel from the source to the destination.
- **Reliability:** The likelihood that a link will fail or perform poorly.
- **Load:** The amount of traffic on a link or network.

Convergence

Convergence refers to the process of all routers in a network updating their routing tables and agreeing on the optimal paths. Faster convergence is preferred, as it minimizes downtime and routing inconsistencies.

Example Algorithms:

- **Dijkstra's Algorithm:** Used by link state protocols like OSPF and IS-IS to find the shortest path.

- **Bellman-Ford Algorithm:** Used by distance vector protocols like RIP to calculate the shortest path.

4.7 Broadcast and Multicast routing

Broadcast and multicast routing are both methods used in computer networks to distribute information to multiple recipients, but they differ in how they achieve this goal.

Broadcast Routing

- **Scope:** Broadcast routing sends data to all devices within a network or subnet. It's a "one-to-all" communication method.
- **Use Case:** Commonly used in local area networks (LANs) to deliver messages that need to reach all devices, such as ARP (Address Resolution Protocol) requests.
- **Efficiency:** Broadcast can lead to network congestion if used excessively because every device in the network must process the broadcasted packet.

Multicast Routing

- **Scope:** Multicast routing is more efficient than broadcast because it sends data only to a specific group of interested recipients. It's a "one-to-many" communication method.
- **Use Case:** Ideal for streaming media, online gaming, and other applications where data needs to be delivered to multiple recipients simultaneously but not to everyone on the network.
- **Efficiency:** Multicast is more efficient in terms of network resource usage because it reduces unnecessary data transmission.

Here's a simple analogy: imagine you're announcing an event. Broadcasting is like using a loudspeaker to announce it to everyone in the city (even those not interested), while multicasting is like sending invitations only to people who signed up for the event.



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

**Faculty of Engineering (Degree)
Semester – 5th**

Subject Name –Computer Networks

Subject Code - BE103404

Faculty Name – Radhika Gokani

Chapter -5

THE LINK LAYER AND LOCAL AREA NETWORKS

5.1 Link Layer in Computer Networks:

The **link layer** (also known as the **data link layer**) is the second layer in the **OSI (Open Systems Interconnection)** model of networking, situated between the physical layer and the network layer. Its primary purpose is to provide reliable communication over a physical link by ensuring proper data transmission and addressing, handling errors, and managing flow control.

Functions of the Link Layer:

- **Framing:** The link layer takes raw data from the network layer (Layer 3) and packages it into frames, which are the units of data transferred over a network.
- **Error Detection and Correction:** The link layer can detect and correct errors that may occur during transmission. Techniques like checksums, cyclic redundancy checks (CRC), and parity checks are used.
- **Addressing:** The link layer uses **MAC (Media Access Control)** addresses for identifying devices on the same physical network segment. This allows for device-to-device communication within the local network.
- **Flow Control:** It manages the rate of data transmission to prevent network congestion or buffer overflow.
- **Access Control:** In networks that use shared media, like Ethernet, the link layer controls when each device can send data, using methods like CSMA/CD (Carrier Sense Multiple Access with Collision Detection).

5.2 LOCAL AREA NETWORK(LAN):

A **Local Area Network (LAN)** is a network of computers and devices that are connected within a relatively small geographic area, such as a home, office, or campus. LANs typically use the link layer protocols for communication and provide high-speed connections between devices.

Characteristics of LANs:

- **Limited Geographic Area:** LANs are typically confined to a small area, such as a building or campus.
- **High Speed:** LANs offer high-speed data transfer rates, typically ranging from 100 Mbps to 10 Gbps or more, depending on the technology used (Ethernet, Wi-Fi, etc.).
- **Private Ownership:** LANs are typically owned and managed by a single organization, such as a company or educational institution.
- **Technologies Used:** Common technologies for building LANs include **Ethernet** (wired LAN) and **Wi-Fi** (wireless LAN).
- **Centralized or Decentralized Setup:** LANs can be structured in different topologies, such as star, bus, ring, or mesh, depending on how the devices are connected.

LAN Protocols and Link Layer:

- **Ethernet:** The most common LAN technology, Ethernet uses the IEEE 802.3 standard. It is a widely-used protocol at the link layer in LANs and defines how devices access the shared medium, detect collisions, and handle data frames.
- **Wi-Fi:** Wireless LANs (WLANs) use the IEEE 802.11 standards. At the link layer, Wi-Fi handles aspects such as wireless signal modulation, error correction, and access control using mechanisms like CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance).
- **Switching:** In LANs, **network switches** operate at the data link layer to forward frames between devices. Switches use MAC addresses to decide where to forward frames, reducing collisions and improving efficiency compared to hubs.

5.3 Relationship Between Link Layer and LANs:

- The **link layer** is integral to the operation of a **Local Area Network** because it ensures that data is correctly framed, transmitted, and addressed to the correct device in the network.
- The **link layer** protocols such as Ethernet or Wi-Fi are the foundation on which the devices in a LAN communicate. Whether using a wired or wireless LAN, the link layer is responsible for delivering data between devices and ensuring reliable communication.

Example: Ethernet in a LAN

Ethernet operates at the **link layer** and is the most common technology used in LANs. Devices in an Ethernet LAN communicate by sending **Ethernet frames**, which contain:

- **Source and Destination MAC Addresses:** To identify the sending and receiving devices on the network.
- **Ethernet Payload:** The data being transmitted.
- **CRC Checksum:** For error detection to ensure data integrity.

In a typical Ethernet-based LAN:

- Devices (computers, printers, switches) are connected through **Ethernet cables** (e.g., Cat 5e or Cat 6).
- **Switches** manage traffic and forward Ethernet frames based on MAC addresses.
- **Routers** may connect the LAN to the internet or other networks, but they operate at higher layers (network layer), while switches and NICs (Network Interface Cards) operate at the link layer.

5.4 INTRODUCTION TO LINK LAYER SERVICES:

The link layer is responsible for several essential tasks that ensure smooth and reliable communication within a local network or between neighboring nodes. It helps devices manage communication over a specific link and provides functionalities that facilitate error-free data transfer and access control.

An overview of the **key services** provided by the **link layer** in data networks:

1. Framing

Framing refers to the process of dividing the data into discrete chunks (called frames) before transmission. A frame typically consists of:

- **Header:** Contains metadata such as source and destination addresses (e.g., MAC addresses), control information, and error detection codes.
- **Payload:** The actual data being transmitted from one device to another.
- **Trailer:** Often used for error detection and correction (e.g., CRC - Cyclic Redundancy Check).

Frames allow data to be transmitted in manageable pieces, and the link layer ensures that the data is properly framed at the sending end and reassembled at the receiving end.

2. Error Detection and Correction

Data transmitted over networks can get corrupted due to various factors such as noise, interference, or collisions. The link layer is responsible for detecting and often correcting such errors. It uses **error detection codes** like:

- **Parity Bit**
- **Checksums**
- **Cyclic Redundancy Check (CRC)**

When a device receives a frame, it performs an error check (usually using CRC) to verify the integrity of the data. If errors are detected, the frame may be discarded or a request for retransmission may be sent to the sender.

3. Flow Control

Flow control refers to mechanisms that manage the rate at which data is sent between devices to prevent network congestion or buffer overflow. If a sender transmits data too quickly, the receiver may become overwhelmed, causing data loss or delays. The link layer manages this by:

- **Controlling the amount of data** sent at any one time.

- **Using techniques** like windowing (in protocols like HDLC or PPP) to manage how much data is in transit at once.
-

4. Medium Access Control (MAC)

The **Medium Access Control (MAC)** sublayer is part of the link layer and is responsible for managing how devices access the shared communication medium (e.g., Ethernet, Wi-Fi). The MAC layer handles the following tasks:

- **Addressing:** Each device on a network has a unique **MAC address**, used to identify it on the local network.
 - **Access Control:** In shared media networks (e.g., wireless or Ethernet), the MAC layer determines when each device is allowed to transmit to avoid collisions. Protocols like **CSMA/CD** (Carrier Sense Multiple Access with Collision Detection) in Ethernet or **CSMA/CA** (Carrier Sense Multiple Access with Collision Avoidance) in Wi-Fi help regulate access to the medium.
-

5. Link Establishment and Termination

The link layer is responsible for establishing and terminating the communication link between devices. This includes:

- **Initialization:** Devices synchronize and set up communication parameters.
 - **Connection Management:** In some protocols, a handshake may be required to confirm the link is active before data transmission begins.
 - **Termination:** Properly closing the connection to free resources after communication is complete.
-

6. Addressing and Device Identification

The link layer provides **local addressing** using **MAC addresses** (Media Access Control addresses) to uniquely identify devices within a local network. Each device connected to a network (e.g., computer, printer, router) has a unique MAC address assigned to its network interface card (NIC).

- **Source MAC address:** Identifies the sender of a frame.
- **Destination MAC address:** Identifies the intended recipient of the frame.

This layer uses the MAC addresses to ensure that data is delivered to the correct destination device on the same local network segment.

7. Link Layer Protocols

Various link-layer protocols provide the services mentioned above. Some of the most common link-layer protocols in data networks include:

- **Ethernet:** A widely used protocol for wired LANs. It operates at the data link layer and defines how frames are structured and transmitted over physical media like coaxial cables or twisted-pair copper cables.

- **Wi-Fi (IEEE 802.11):** Used for wireless communication in Local Area Networks (WLANs). It uses MAC addressing and operates with a different set of rules to manage wireless communication.
 - **PPP (Point-to-Point Protocol):** Often used in direct communication between two devices, like a computer and a router, over a serial link.
 - **HDLC (High-Level Data Link Control):** A protocol used for point-to-point communication and often employed in WANs (Wide Area Networks).
-

8. Reliable Data Transfer (Optional)

While the link layer ensures reliable transmission within a local network, in some cases, **reliable data transfer** protocols such as **ARQ (Automatic Repeat reQuest)** can be implemented. These protocols ensure that data is successfully delivered by requesting retransmissions in case of packet loss or error detection.

5.6 ERROR DETECTION AND ERROR CORRECTION TECHNIQUE:

Error detection techniques are employed to detect errors that may have occurred during the transmission of data. The key idea is to add redundancy to the data and then check if the transmitted data matches the expected format upon arrival.

the **common error detection techniques** used in computer networks:

1. Parity Bit

A **parity bit** is a simple error detection technique used to check the integrity of data. It adds a single bit (0 or 1) to the end of a data unit (like a byte) to make the number of 1-bits either even or odd, depending on the type of parity.

- **Even Parity:** The number of 1s in the data + parity bit is even.
- **Odd Parity:** The number of 1s in the data + parity bit is odd.

Limitations:

- It can only detect single-bit errors.
- It cannot correct errors or detect errors involving an even number of bit flips.

2. Checksums

A **checksum** is a value computed from a data packet and sent along with the data. At the receiving end, the same checksum algorithm is applied to the received data, and if the calculated checksum matches the transmitted one, the data is assumed to be error-free.

The checksum is typically calculated by summing up the bits in a data packet and storing the sum (possibly with modulo operations to fit within a fixed number of bits). For example, in **Internet Protocol (IP)**, the checksum is calculated over the header.

Limitations:

- It can detect errors, but it's not as powerful as more complex error detection techniques like CRC.
- It is less effective in detecting errors with large data sets.

3. Cyclic Redundancy Check (CRC)

Cyclic Redundancy Check (CRC) is a more advanced and powerful error detection method that is widely used in computer networks (e.g., Ethernet, CD/DVD data transmission, and file storage systems). CRC uses polynomial division to generate a fixed-size checksum for the transmitted data.

Steps involved in CRC:

1. **Polynomial Division:** Data is treated as a large binary number, and it is divided by a pre-agreed divisor (a generator polynomial).
2. **Remainder:** The remainder of this division process becomes the CRC checksum.
3. The receiver performs the same division and compares the calculated remainder with the received CRC checksum. If they match, the data is assumed to be error-free.

Advantages:

- Very effective in detecting common transmission errors such as burst errors.
- It is used in a wide range of protocols (Ethernet, Frame Relay, ATM).

Limitations:

- Like checksums, CRC is only for error detection and does not correct errors by itself.

Error Correction Techniques in Computer Networks

Error correction goes a step beyond detection by not only identifying errors but also attempting to fix them without needing a retransmission. This is especially useful in situations where retransmission is costly, or real-time communication is necessary (e.g., voice or video calls).

some **common error correction techniques** used in computer networks:

1. Automatic Repeat reQuest (ARQ)

ARQ is a widely-used error control mechanism in which the receiver requests the sender to retransmit a data packet if an error is detected. There are several types of ARQ, depending on how errors and retransmissions are handled:

- **Stop-and-Wait ARQ:** After sending each data packet, the sender waits for an acknowledgment (ACK) from the receiver before sending the next packet. If no ACK is received or if an error is detected, the sender retransmits the packet.
- **Go-Back-N ARQ:** The sender can send multiple packets before waiting for an acknowledgment. If an error is detected in any packet, the receiver discards all

subsequent packets, and the sender retransmits the erroneous packet and all subsequent ones.

- **Selective Repeat ARQ:** Similar to Go-Back-N, but only the specific erroneous packet is retransmitted, which improves efficiency.

Advantages:

- Simple and effective, especially in reliable communication environments like TCP/IP.
- Helps maintain data integrity in unreliable networks.

Limitations:

- Requires retransmission of data, which can introduce delays and reduce bandwidth efficiency.
- Not suitable for real-time applications due to retransmission delays.

2. Forward Error Correction (FEC)

Forward Error Correction (FEC) is a technique in which the sender encodes the data with redundant information so that the receiver can detect and correct errors without requiring a retransmission.

The redundancy is achieved by adding **extra bits** (error-correcting codes) to the transmitted message. Common FEC codes include:

- **Hamming Code:** Adds redundant bits to the data to allow the detection and correction of single-bit errors.
- **Reed-Solomon Codes:** Widely used in systems like CDs, DVDs, and satellite communication, these codes can correct multiple errors in the transmitted data.

How FEC Works:

1. The sender adds extra error-correcting bits to the message (e.g., by applying a code like Hamming or Reed-Solomon).
2. The receiver uses these extra bits to check for errors and correct them without needing a retransmission.

Advantages:

- Effective for high-latency or lossy networks (e.g., satellite communication).
- Reduces the need for retransmissions, making it suitable for real-time applications like video streaming and VoIP.

Limitations:

- It requires additional bandwidth because of the redundant information.
- The error-correction capability is finite, and it can only correct errors up to a certain extent (depending on the code used).

3. Hybrid ARQ (HARQ)

Hybrid ARQ (HARQ) combines the benefits of both **ARQ** and **FEC**. In HARQ, the receiver uses FEC to correct some errors but requests a retransmission for those that can't be corrected. The receiver may also combine received versions of a packet to improve the chances of error correction.

- **Type I HARQ:** Combines ARQ with simple error detection/correction codes like checksums or CRCs.
- **Type II HARQ:** Combines ARQ with FEC codes (e.g., Reed-Solomon or Turbo codes).

Advantages:

- Improves network throughput and reduces the need for retransmissions compared to standard ARQ.
- Useful in wireless networks with high error rates, such as mobile communication systems (e.g., LTE, 5G).

5.7 MULTIPLE ACCESS PROTOCOLS:

Multiple access protocols are used to manage how multiple devices share a common communication medium (such as a wire or wireless spectrum) in a way that prevents data collisions and ensures efficient transmission. These protocols define how devices access the medium, communicate, and handle potential conflicts when multiple devices try to send data simultaneously. The primary goal is to ensure fair and efficient use of the shared resource.

Types of Multiple Access Protocols

The most common **multiple access protocols** used in computer networks:

1. Aloha Protocol

The **Aloha protocol** was one of the first multiple access protocols, initially developed for satellite communication. It is a simple and foundational protocol for random access in a shared medium.

- **Pure Aloha:** Devices transmit data at any time. If there is a collision (i.e., another device transmits at the same time), the sender must wait a random time before retrying the transmission.
 1. **Pros:** Simple, easy to implement.
 2. **Cons:** Inefficient, as the likelihood of collision is high, especially as the number of devices increases. The throughput is limited to about 18.4% of the maximum potential capacity.
 - **Slotted Aloha:** The time is divided into fixed-length slots. Devices are only allowed to transmit at the beginning of a time slot. This reduces the chance of collisions since the devices are synchronized to some extent.
 1. **Pros:** More efficient than pure Aloha, with a throughput of about 36.8%.
 2. **Cons:** Still subject to collisions, and the slots can cause delays if the devices have less data to send.
-

2. Carrier Sense Multiple Access (CSMA)

Carrier Sense Multiple Access (CSMA) is a protocol where devices **listen to the medium** before transmitting data to ensure that no other device is currently sending data. It reduces collisions by ensuring that devices only transmit when the channel is clear.

- **Non-persistent CSMA:** A device that detects that the channel is busy waits for a random time before checking again to see if the channel is idle.
- **1-persistent CSMA:** The device keeps checking the channel, and as soon as it detects that the channel is idle, it immediately starts transmitting.
- **P-persistent CSMA:** Used in time-slotted channels. If the channel is idle, a device will transmit with probability **p**; if the channel is busy, it waits for the next time slot.

Pros:

- Simple and efficient for low traffic.
- Reduces the need for synchronization.

Cons:

- **Collisions** may still occur, especially in high-traffic scenarios.
- **Collision Detection** (CSMA/CD) or **Collision Avoidance** (CSMA/CA) may be needed to further reduce the impact of collisions.

3. Carrier Sense Multiple Access with Collision Detection (CSMA/CD)

CSMA/CD is an extension of the **CSMA** protocol used in Ethernet networks, where devices not only listen to the medium (Carrier Sense) but also detect if a collision occurs during transmission (Collision Detection).

- **How CSMA/CD Works:**
 1. The device checks if the medium is idle before transmitting (Carrier Sense).
 2. If the medium is idle, the device starts transmitting.
 3. While transmitting, the device listens for a collision. If a collision occurs, both devices stop transmitting immediately, wait a random time, and retransmit their data.

Pros:

- More efficient than basic CSMA as it minimizes the duration of collisions.
- **Ethernet** networks widely use this protocol, making it suitable for local area networks (LANs).

Cons:

- **Collisions** still occur, but the network recovers quickly.
- In highly congested networks, **collisions** can reduce efficiency.

4. Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)

CSMA/CA is an extension of **CSMA** primarily used in **wireless networks** (e.g., Wi-Fi). Since it is difficult to detect collisions in wireless communication (due to the **hidden node problem**), **CSMA/CA** focuses on **avoiding collisions** rather than detecting them.

- **How CSMA/CA Works:**

1. The device listens to the channel (Carrier Sense) to check if it's idle.
2. If idle, the device waits for a short **backoff period** before transmitting (randomized).
3. Before transmission, the sender sends a **Request to Send (RTS)** message.
4. If the channel is clear, the receiver sends a **Clear to Send (CTS)** message, which informs other devices to defer their transmission.
5. The sender then sends the actual data.

Pros:

- Helps to avoid collisions, especially in wireless environments.
- **Backoff** periods reduce the likelihood of collisions.

Cons:

- Higher overhead due to additional signaling (RTS/CTS).
 - **Hidden node problem** can still occur if devices cannot hear each other.
-

5. Time Division Multiple Access (TDMA)

TDMA divides the time on the shared communication medium into time slots. Each device is assigned a specific time slot to transmit its data. This eliminates collisions because only one device can transmit in a given time slot.

- **How TDMA Works:**

1. The available bandwidth is divided into fixed-size time slots.
2. Each device is allocated a time slot during which it can send data.
3. Devices can transmit in their time slots without worrying about collisions from other devices.

Pros:

- Efficient use of bandwidth and reduced collisions.
- Suitable for systems with predictable traffic loads.

Cons:

- Requires synchronization between devices.
 - Not suitable for bursty traffic, as unused slots must remain idle.
-

6. Frequency Division Multiple Access (FDMA)

In **FDMA**, the communication bandwidth is divided into **multiple frequency bands**. Each device is assigned a specific frequency band in which it can transmit its data.

- **How FDMA Works:**

1. The available frequency spectrum is divided into several smaller frequency channels.
2. Each device is assigned a unique frequency channel for communication.
3. Devices transmit simultaneously but on different frequencies.

Pros:

- Allows simultaneous transmission by multiple devices without collision.
- Used in systems like cellular networks.

Cons:

- **Limited efficiency** as frequency bands may be underutilized.
 - Requires precise frequency management and coordination.
-

7. Code Division Multiple Access (CDMA)

CDMA uses **unique codes** to distinguish between transmissions from different devices. All devices transmit simultaneously, but each device's data is encoded with a unique code, allowing receivers to distinguish between different signals based on their code.

- **How CDMA Works:**

1. Each device is assigned a unique spreading code.
2. All devices transmit at the same time over the same frequency channel.
3. The receiver uses the unique code to extract the correct data from the mixed signals.

Pros:

- **Efficient** use of bandwidth.
- Allows simultaneous transmission by multiple devices without interference if codes are chosen properly.

Cons:

- Requires sophisticated signal processing at both the transmitter and receiver.
- Limited by the number of available codes.

5.8 ADDRESSING:

Addressing in Computer Networks (CN) refers to the method by which devices (computers, routers, servers, etc.) are identified on a network, enabling communication between them. Addresses are used to specify the source and destination of data packets transmitted across the network. The type of addressing used depends on the kind of network (e.g., local area network (LAN), wide area network (WAN), or the Internet), the communication protocol (e.g., TCP/IP), and the layer in the OSI model that is involved.

Types of Addressing in Computer Networks

Physical Addressing (MAC Address)

1. **Definition:** A **Physical Address**, also known as the **MAC (Media Access Control) address**, is a unique hardware identifier assigned to the network interface card (NIC) of each device.
2. **Used in:** The **Data Link Layer** of the OSI model.
3. **Format:** A MAC address is usually a 48-bit address (6 bytes) represented as a hexadecimal string, such as 00:1A:2B:3C:4D:5E.
4. **Purpose:** It uniquely identifies a device on a local network, allowing devices to communicate within the same physical network segment or LAN (Local Area Network).
5. **Example:** When a device sends a frame to another device on a local network, it uses the recipient's MAC address as the destination address.

Pros:

1. Unique to the device (burned into hardware).
2. Essential for communication within a local network.

Cons:

1. Not suitable for communication over large or global networks, such as the Internet, because MAC addresses are only valid within the local network.

Logical Addressing (IP Address)

1. **Definition:** A **logical address** is an address assigned to a device on a network that is used to route data between different networks. The most common logical address is the **IP address (Internet Protocol)**.
2. **Used in:** The **Network Layer** of the OSI model.
3. **Format:**
 1. **IPv4:** A 32-bit address written in four decimal numbers separated by periods (e.g., 192.168.1.1).
 2. **IPv6:** A 128-bit address written in eight groups of four hexadecimal digits separated by colons (e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334).
4. **Purpose:** Logical addresses allow communication across multiple networks, such as the Internet. They help route data between devices over a network and can be hierarchical, meaning they can be used to identify networks and subnets as well as individual devices.
5. **Example:** When data is sent from one network to another, routers use IP addresses to forward the data packets across different networks.

Pros:

1. Facilitates routing of data across the global network.
2. Supports addressing for both large and small networks.

Cons:

1. IP addresses can change depending on the network and may require reconfiguration or assignment by a DHCP server.

Port Addressing (Port Numbers)

1. **Definition: Port addressing** involves the use of port numbers to identify specific applications or services running on a device.
2. **Used in:** The **Transport Layer** of the OSI model (specifically for protocols like **TCP** and **UDP**).
3. **Format:** Port numbers are 16-bit numbers, typically ranging from 0 to 65535.
 1. Ports **0-1023** are known as **well-known ports** (e.g., HTTP uses port 80, HTTPS uses port 443).
 2. Ports **1024-49151** are **registered ports**, used by user applications.
 3. Ports **49152-65535** are **dynamic or private ports**.
4. **Purpose:** Port numbers are used to direct data packets to specific services or applications on a device (e.g., web servers, email servers).
5. **Example:** When a web browser communicates with a web server, it typically uses port 80 (HTTP) or 443 (HTTPS) for communication.

Pros:

1. Allows multiple applications to share the same IP address (via different ports).
2. Essential for identifying services on a device.

Cons:

1. Port numbers can be susceptible to conflicts if not managed properly.
2. Applications can be vulnerable to security attacks if port management is not secure.

Application Layer Addressing (Domain Names)

1. **Definition:** At the **Application Layer**, addressing is done using **domain names**. Domain names are human-readable names that map to IP addresses.
2. **Used in:** The **Application Layer** (DNS - Domain Name System).
3. **Format:** Domain names are structured hierarchically, such as `www.example.com`.
4. **Purpose:** Domain names are used to access websites and services on the Internet, and DNS translates these domain names into IP addresses (known as **DNS resolution**).
5. **Example:** When you type `www.google.com` into a browser, DNS resolves it to an IP address (e.g., `142.250.182.14`), and the browser connects to that address.

Pros:

1. Easy for users to remember.
2. Provides a more user-friendly way of accessing websites than memorizing IP addresses.

Cons:

1. Dependent on DNS servers for resolution.
 2. If DNS servers are down or misconfigured, accessing resources can be difficult.
-

Multicast Addressing

1. **Definition: Multicast addressing** allows data to be sent to multiple recipients (a group of devices) using a single address.
2. **Used in:** The **Network Layer** (IP multicast) or **Data Link Layer** (MAC multicast).
3. **Format:**
 1. **IPv4 Multicast:** A special range of IP addresses from 224.0.0.0 to 239.255.255.255.
 2. **IPv6 Multicast:** Addresses beginning with FF00::/8.
4. **Purpose:** Multicast allows a single device to send data to a specific group of devices rather than broadcasting to the entire network. It's used for applications like live streaming, video conferencing, and group communications.
5. **Example:** A video streaming service might send a multicast stream to multiple devices in a network without needing to send separate streams to each individual device.

Pros:

1. Efficient use of bandwidth, especially in group communication scenarios.
2. Reduces network load compared to sending individual copies of data to each recipient.

Cons:

1. Complex to manage and configure.
 2. Not all networks support multicast traffic.
-

Special Address Types in IP Addressing**Private IP Addresses**

1. Private IP addresses are reserved for use within private networks. They cannot be routed over the public internet. The ranges for private IP addresses in IPv4 are:
 1. 10.0.0.0 - 10.255.255.255
 2. 172.16.0.0 - 172.31.255.255
 3. 192.168.0.0 - 192.168.255.255

Public IP Addresses

1. Public IP addresses are assigned to devices that need to be accessed from outside a private network. These are routable over the internet and are allocated by organizations like **ICANN** (Internet Corporation for Assigned Names and Numbers).

Loopback Address

1. The **loopback address** (commonly 127.0.0.1 in IPv4 or ::1 in IPv6) is used to test network functionality within the same machine. It allows the device to send and receive data to itself.

Broadcast Address

1. A **broadcast address** is used to send data to all devices on a network. In IPv4, a network's broadcast address is typically the highest address in the subnet (e.g., 192.168.1.255 for a subnet 192.168.1.0/24).

5.9 ETHERNET:

Ethernet is one of the most widely used **local area network (LAN)** technologies in computer networks. It is a set of standards that defines the physical and data link layers of the **OSI model**, and it specifies how devices on a network communicate with each other. Ethernet is used in both wired and wireless networks (though the traditional Ethernet is wired), and it has evolved over the years to support faster data rates and improved reliability.

Key Aspects of Ethernet in Computer Networks **Ethernet Standards and Evolution**

1. **Original Ethernet (10BASE-T)**: The first Ethernet standard was developed in the 1970s by Xerox, and it was standardized by the IEEE (Institute of Electrical and Electronics Engineers) as **IEEE 802.3**. The original Ethernet supported speeds of **10 Mbps** over coaxial cables.
2. **Fast Ethernet (100BASE-T)**: In the 1990s, **Fast Ethernet** was introduced, which increased the speed to **100 Mbps** and used twisted-pair copper cables (Category 5, or Cat 5).
3. **Gigabit Ethernet (1000BASE-T)**: The next evolution was **Gigabit Ethernet**, which supported speeds of **1 Gbps** and was commonly used for high-performance LANs.
4. **10-Gigabit Ethernet (10GBASE-T)**: As demands for faster speeds increased, **10-Gigabit Ethernet** became widely adopted, allowing speeds of **10 Gbps** over copper and fiber optic cables.
5. **Higher-Speed Ethernet**: More recent standards support even faster Ethernet speeds, such as **40 Gbps**, **100 Gbps**, and beyond, using fiber optic cables.

Ethernet Frame Structure:

Ethernet uses a **frame** to transmit data across the network. Each Ethernet frame contains a structure with several fields, each serving a specific purpose in the communication process.

1. **Preamble:** 7 bytes used to synchronize the transmission and ensure the receiver is ready to read the frame.
 2. **Start Frame Delimiter (SFD):** 1 byte that marks the beginning of the frame.
 3. **Destination MAC Address:** 6 bytes that identify the recipient device (this is the physical address).
 4. **Source MAC Address:** 6 bytes that identify the sender's device.
 5. **Type/Length:** 2 bytes indicating the type of protocol being carried (e.g., IP, ARP) or the length of the data field.
 6. **Data/Payload:** The actual data being transmitted. Its size can vary but is usually between 46 and 1500 bytes.
 7. **Padding:** If the data is smaller than 46 bytes, padding is added to make the frame size at least 64 bytes (the minimum size).
 8. **Frame Check Sequence (FCS):** 4 bytes that contain a **CRC (Cyclic Redundancy Check)** to detect errors in the frame.
-

Ethernet Media Types:

Ethernet operates over various physical media, including:

1. **Twisted Pair Cables (Copper):** Used in most modern Ethernet networks, supporting speeds from 10 Mbps (10BASE-T) up to 10 Gbps (10GBASE-T).
 2. **Fiber Optic Cables:** Used for high-speed long-distance connections, supporting speeds of 1 Gbps to 100 Gbps or more (e.g., **1000BASE-LX**, **10GBASE-SR**).
 3. **Coaxial Cable:** Used in the early versions of Ethernet (e.g., **10BASE-2** and **10BASE-5**), but now largely obsolete.
-

Ethernet Switching

Modern Ethernet networks often use **Ethernet switches** to connect multiple devices within a LAN. A switch is a **network device** that connects devices based on their **MAC addresses**, making decisions about where to forward data frames.

1. **Switching Process:** When a device sends a frame to another device on the same network, the switch examines the destination MAC address. If the switch knows which port the device is connected to (from its MAC address table), it forwards the frame directly to that port.
 2. **Broadcasting:** If the switch doesn't know the destination address (i.e., it's not in the MAC address table), it will broadcast the frame to all connected devices, and the intended recipient will process the frame.
 3. **Full Duplex vs Half Duplex:** Ethernet used to be **half-duplex**, meaning data could flow in only one direction at a time. Modern switches support **full-duplex** communication, where data can flow simultaneously in both directions, improving efficiency and reducing collisions.
-

Ethernet Protocols

Ethernet typically operates in conjunction with several higher-layer protocols:

1. **IP (Internet Protocol):** Ethernet is often used to carry **IP packets**. Devices on a network communicate using **IP addresses** to route data, and Ethernet provides the physical addressing through **MAC addresses**.
 2. **ARP (Address Resolution Protocol):** ARP is used to map **IP addresses** to **MAC addresses** in a local network. This allows devices to know the MAC address of a device they want to communicate with.
 3. **TCP/UDP:** Once the data is carried over Ethernet, higher-level protocols like **Transmission Control Protocol (TCP)** or **User Datagram Protocol (UDP)** are used for reliable or best-effort communication, respectively.
-

Ethernet Topologies

Ethernet can operate in various topologies, but the most common ones are:

1. **Bus Topology:** In early Ethernet systems, a single coaxial cable was used to connect all devices in a line. This is now largely obsolete but was the original Ethernet topology.
 2. **Star Topology:** The most common topology today, where each device is connected to a central Ethernet switch. This provides better performance and fault tolerance compared to bus topology.
 3. **Ring Topology:** Used in some legacy networks but not typical in Ethernet. It involves connecting devices in a circular path. Ethernet protocols such as **Token Ring** were used in the past, but Ethernet now predominates in LANs.
-

Ethernet MAC Addressing and Collision Domains

1. **MAC Addresses:** As mentioned earlier, each Ethernet device has a unique **MAC address** assigned to it. This address is used to identify the source and destination of data frames within the local network.
 2. **Collision Domains:** In traditional Ethernet networks using hubs, all devices in the same physical network share the same collision domain. When two devices send data at the same time, a collision occurs, and both devices must retransmit their data.
 1. **Modern Ethernet switches** have replaced hubs and reduce collision domains by creating separate collision domains for each port. This means that devices connected to different ports on a switch can send data simultaneously without collisions.
-

Ethernet in Modern Networks

1. **Gigabit Ethernet and Beyond:** Ethernet has evolved to support higher speeds such as **Gigabit Ethernet (1000BASE-T)** and **10-Gigabit Ethernet (10GBASE-T)**, making it suitable for modern high-speed data centers, enterprise networks, and even **home networks**.
2. **Power over Ethernet (PoE):** Modern Ethernet standards support **Power over Ethernet (PoE)**, which allows devices like **IP cameras**, **wireless access points**, and **VoIP phones** to receive both data and power over the same Ethernet cable. This reduces the need for separate power supplies.

Advantages of Ethernet

- **Scalability:** Ethernet can support a large number of devices with minimal configuration.
 - **Cost-effective:** Ethernet technology is relatively inexpensive compared to other networking technologies.
 - **High-speed data transfer:** With speeds ranging from 10 Mbps to 400 Gbps or more, Ethernet provides robust data transfer capabilities.
 - **Compatibility:** Ethernet is widely used and is supported by most networking hardware and devices.
 - **Ease of installation and maintenance:** Ethernet networks are straightforward to install, and the technology is well-documented, making troubleshooting easier.
-

Disadvantages of Ethernet

- **Limited range:** Ethernet over copper (e.g., Cat 5e, Cat 6 cables) is limited to relatively short distances (100 meters). Fiber optics can extend the range significantly, but it's more expensive.
- **Collision in shared media:** In older Ethernet systems (using hubs), collisions can occur, reducing efficiency. However, this is mostly mitigated by using switches in modern networks.
- **Interference in wired networks:** Ethernet using copper cables can experience electrical interference, which can degrade performance. Fiber optic cables, on the other hand, are immune to this.

5.10 VLAN:

VLAN (Virtual Local Area Network) is a logical grouping of devices within a network that are segmented into smaller broadcast domains, even if they are physically located on different switches. VLANs allow for better network management, enhanced security, and optimized traffic flow, even when devices are spread across different geographical locations.

In a traditional LAN, all devices connected to the same switch belong to the same broadcast domain, meaning that any broadcast sent by a device is received by all other devices in the same domain. A VLAN breaks this constraint by segmenting the network into multiple smaller, isolated broadcast domains. Devices in different VLANs do not communicate directly unless routing is involved, even if they are physically connected to the same network infrastructure.

Key Concepts of VLAN in Computer Networks

VLAN Identification

1. Each VLAN is assigned a unique identifier called a **VLAN ID**, which is a number between **1 and 4095** (in the IEEE 802.1Q standard).
2. VLAN IDs **1-1005** are reserved for standard use, while **1006-4095** are typically used for private VLANs or by service providers.

VLAN Tagging (IEEE 802.1Q)

1. **VLAN tagging** involves adding a 4-byte tag to the Ethernet frame to specify the VLAN to which the frame belongs. This tag is added by the **network switch** as it forwards the data.
2. **802.1Q Tag**: The 4-byte tag consists of:
 1. **Tag Protocol Identifier (TPID)**: 2 bytes, indicating that the frame is VLAN-tagged.
 2. **Priority Code Point (PCP)**: 3 bits, used for Quality of Service (QoS) marking.
 3. **VLAN ID**: 12 bits, identifying the VLAN to which the frame belongs.
 4. **CFC (Canonical Format Indicator)**: 1 bit.
3. This tag is inserted between the source MAC address and the Ethertype field in the Ethernet frame.

Types of VLANs There are several types of VLANs, based on their purpose and functionality:

1. **Data VLAN (User VLAN)**: This is the most common type of VLAN, used for user devices like computers, printers, and servers. It carries regular network traffic.
2. **Voice VLAN**: Used to prioritize voice traffic (e.g., for **VoIP** phones), ensuring low latency and high-quality communication. This VLAN has higher priority over others.
3. **Management VLAN**: Dedicated to network management traffic, such as communication with routers, switches, or other network devices. It is used for administrative access to network equipment.
4. **Native VLAN**: In an 802.1Q trunk link, the **native VLAN** is the VLAN that is not tagged. It is used for untagged traffic and typically represents the default VLAN (usually VLAN 1).
5. **Storage VLAN**: Used for network storage traffic, such as **iSCSI** or **NAS** (Network Attached Storage).
6. **Private VLAN (PVLAN)**: Used to isolate devices within the same VLAN. This allows for strict isolation while still being part of a broader VLAN configuration.

VLAN Trunking

1. VLAN trunking allows multiple VLANs to be carried over a single physical link between switches, routers, or other network devices. The trunk link is configured to carry traffic from multiple VLANs simultaneously.
2. The **IEEE 802.1Q standard** is most commonly used for VLAN trunking. It involves tagging frames with the VLAN ID, allowing the receiving device to know which VLAN the traffic belongs to.
3. A trunk link can be configured as **tagged** (frames from multiple VLANs) or **untagged** (usually associated with the native VLAN).

VLAN Routing (Inter-VLAN Routing)

1. Devices within different VLANs cannot communicate directly with each other. To enable communication between VLANs, **Inter-VLAN routing** is needed.
2. This is typically accomplished by a **router** or a **Layer 3 switch**. These devices route traffic between VLANs based on IP addresses.
3. In some setups, a **Router on a Stick** configuration can be used, where a single router interface is configured with subinterfaces for each VLAN.

VLAN Configuration

1. VLANs are typically configured on **network switches**. The configuration usually involves:
 1. Creating VLANs on a switch (e.g., VLAN 10 for the HR department, VLAN 20 for the IT department).
 2. Assigning switch ports to specific VLANs (e.g., connecting devices to a particular VLAN port).
 3. Configuring trunk links between switches to carry multiple VLANs across the network.

Benefits of VLANs

1. **Improved Security:** VLANs provide isolation between different groups of users or devices, reducing the risk of unauthorized access. For example, sensitive data can be placed in a separate VLAN, isolated from the rest of the network.
2. **Reduced Broadcast Traffic:** By segmenting the network into multiple VLANs, broadcast traffic is contained within the VLAN. This prevents unnecessary traffic from propagating throughout the entire network.
3. **Network Management:** VLANs make it easier to manage large networks. Administrators can group devices based on function or department, regardless of their physical location.
4. **Simplified Network Design:** VLANs simplify network design and allow for more flexible topology. Devices that need to communicate with each other can be placed in the same VLAN, even if they are physically spread out over multiple switches or floors.
5. **Better Performance:** By limiting the scope of broadcast traffic, VLANs can improve network performance and reduce congestion.

Challenges with VLANs

1. **Complexity:** Configuring and managing VLANs can be more complex, especially in larger networks.
2. **Inter-VLAN Routing:** Communication between VLANs requires additional configuration, such as configuring routers or Layer 3 switches for inter-VLAN routing.
3. **Broadcast Storms:** Although VLANs reduce broadcast traffic, if misconfigured, a large number of VLANs could result in broadcast storms that affect performance.
4. **VLAN Hopping:** In some cases, attackers may exploit misconfigurations to bypass VLAN segmentation. Proper network security measures must be in place to prevent such attacks.

VLAN Examples and Use Cases

Simple Office Network

1. **VLAN 10:** Sales Department
2. **VLAN 20:** HR Department
3. **VLAN 30:** IT Department
4. Devices in each department are placed in their respective VLANs to isolate traffic between departments.

Voice and Data Network

1. **VLAN 100:** Data VLAN (for computers, printers, etc.)
2. **VLAN 200:** Voice VLAN (for VoIP phones)
3. VoIP phones are placed in their own VLAN to ensure priority for voice traffic, preventing delays or poor-quality calls that could occur due to network congestion from data traffic.

Multi-site Company

1. **VLAN 10:** Headquarters
2. **VLAN 20:** Remote Office 1
3. **VLAN 30:** Remote Office 2
4. Each office has its own VLAN, and traffic from these offices can be routed through a central router, ensuring secure and efficient communication.